

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Оюутны код: s21c011b

Оюутны овог нэр: Үдэлгомбо ТАМИР

“CROWDSOURCE” ХҮРГЭЛТИЙН НЭГДСЭН СИСТЕМ

/ДЭД БАКАЛАВРЫН ТӨГСӨЛТИЙН СУДАЛГААНЫ АЖИЛ/

Удирдагч багш

/Н.Соронзонболд, Магистр/

Гүйцэтгэсэн оюутан

/s21c011b, Ү. Тамир/

Улаанбаатар хот

2026 он

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ
КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Дэд бакалаврын төгсөлтийн судалгааны ажил (061301)

“CROWDSOURCE” ХҮРГЭЛТИЙН НЭГДСЭН СИСТЕМ

Удирдагч багш

/Н.Соронзонболд, Магистр/

Гүйцэтгэсэн оюутан

/s21c011b, Ү. Тамир/

Улаанбаатар хот

2026 он

“Crowdsource” хүргэлтийн нэгдсэн систем

Компьютерын ухааны тэнхим

Ү. Тамир

Хураангуй

Монгол Улсын жижиг, дунд бизнесүүд хоорондоо бараа захиалах, хүргэх үйлээ гол төлөв утас, мессеж, цаасаар гүйцэтгэсээр байна. Ийм гараар зохицуулдаг хэлбэр нь цаг алдагдуулж, алдаа үүсгэж, явцыг хянахад төвөгтэй болгож байна.

Төсөлд зуучлагч оператор шаардахгүй, олон байгууллага зэрэг үйлчлүүлдэг, хүргэгчид өөрсдөө даалгавраа сонгодог B2B хүргэлтийн нэгдсэн платформыг боловсруулсан. Систем нь хүргэгчдэд зориулсан гар утасны аппликейшн, борлуулагчдад зориулсан веб аппликейшн, үүлэн дээр ажилладаг backend гэсэн гурван үндсэн хэсгээс бүрдэнэ.

Системийн найдвартай байдал, аюулгүй байдал, хурдны үзүүлэлтийг туршилтаар үнэлсэн. Нэг даалгаврыг олон хүргэгч зэрэг авахаар өрсөлдөхөд давхардал үүсэхгүй, байгууллагуудын мэдээлэл бие биендээ алдагдахгүй, хүргэлтийг баталгаажуулах код нь аюулгүй ажиллаж байгаа нь нотлогдсон.

Судалгааны үр дүнд тус систем нь Монголын жижиг, дунд бизнесийн хүргэлтийн гол бэрхшээлүүдийг бодитоор шийдэж чадах нь батлагдлаа.

Товчлолын жагсаалт

ЖДБ	Жижиг, дунд бизнес
B2B	Business to Business
API	Application Programming Interface
BaaS	Backend-as-a-Service
BFF	Backend-for-Frontend
DB	Database
ERD	Entity Relationship Diagram
RLS	Row Level Security
RBAC	Role-Based Access Control
RPC	Remote Procedure Call
CRUD	Create, Read, Update, Delete
SQL	Structured Query Language
JWT	JSON Web Token
UUID	Universally Unique Identifier
POD	Proof of Delivery
ePOD	electronic Proof of Delivery
OTP	One-Time Password
KYC	Know Your Customer
FCFS	First-Come, First-Served
MVCC	Multi-Version Concurrency Control
WAL	Write-Ahead Log
SDK	Software Development Kit
SLA	Service Level Agreement
MVP	Minimum Viable Product
SSR	Server-Side Rendering
ISR	Incremental Static Regeneration
VU	Virtual User (k6 load testing)
RTT	Round-Trip Time
E2E	End-to-End
CI	Continuous Integration

Ажлын төлөвлөгөө

Оюутны нэр: Ү. Тамир

Удирдагч багшийн нэр: Н.Соронзонболд

Судалгааны ажлын сэдэв: “Crowdsourcing” хүргэлтийн нэгдсэн систем

Сар	Долоо хоног	Төлөвлөгөө	Гүйцэтгэл
10 сар	1	Сэдэв сонгох, B2B хүргэлтийн зах зээлийн судалгаа эхлэх	✓
	2	Last-mile логистик, crowdsourced хүргэлтийн онолын судалгаа	✓
	3	Олон улсын платформ судалгаа (Uber Freight)	✓
	4	Дотоодын платформ судалгаа (Para Logistics), харьцуулалт	✓
11 сар	1	Системийн архитектур төлөвлөх, технологийн стек сонгох	✓
	2	Өгөгдлийн сангийн ERD загвар боловсруулах (11 хүснэгт)	✓
	3	Supabase төсөл үүсгэх, PostgreSQL хүснэгтүүд хөгжүүлэх	✓
	4	RLS бодлого, триггер, серверийн функцүүд хэрэгжүүлэх	✓
12 сар	1	TCA Үзлэг 1	✓
	2	Supabase Auth, Storage тохиргоо, KYC баталгаажуулалтын процесс	✓
	3	Хүргэгчийн мобайл апп (React Native/Expo) суурь бүтэц	✓
	4	Бүртгэл, нэвтрэлт, профайл удирдлагын дэлгэцүүд	✓
1 сар	1	Хүргэгчийн апп: нээлттэй даалгавар, даалгавар авах функц	✓
	2	Хүргэгчийн апп: хүргэлт гүйцэтгэх, орлого хянах	✓
	3	Борлуулагчийн веб апп (Next.js) суурь бүтэц, бүтээгдэхүүн удирдлага	✓
	4	Веб апп: захиалга удирдлага, хүргэлтийн даалгавар үүсгэх	✓
2 сар	1	Бодит цагийн мэдэгдэл, нэгтгэлийн туршилт	✓
	2	Аюулгүй байдлын тест, зэрэгцээ хандалтын тест	✓
	3	Тайлангийн 1-р бүлэг (Удиртгал) бичих	✓
	4	Тайлангийн 2-р бүлэг (Судалгаа, онол) бичих	✓
3 сар	1	TCA Үзлэг 2	✓
	2	Тайлангийн 3-р бүлэг (Арга зүй, диаграмм) бичих	✓
	3	Тайлангийн 4-р бүлэг (Дүгнэлт) бичих	✓
	4	Санал хүсэлтийн дагуу засвар хийх, код баримтжуулалт	✓
4 сар	1	График, хүснэгт, диаграммуудыг эцэслэх	✓
	2	Ном зүй, хавсралт, товчлол бэлдэх	✓
	3	Хамгаалалтын слайд бэлдэх, дадлага хийх	✓
	4	TCA урьдчилсан хамгаалалт	□
5 сар	1	TCA урьдчилсан хамгаалалт	□
	2	Урьдчилсан хамгаалалтын санал тусгах, засвар хийх	□
	3	Слайд сайжруулах, эцсийн бэлтгэл	□
	4	TCA үндсэн хамгаалалт	□

Гүйцэтгэлийн төлөв шинэчлэгдсэн огноо: 2026-04-19.

Удирдагч багш:

/Н.Соронзонболд/

Гүйцэтгэсэн оюутан:

/s21c011b, Ү. Тамир/

Гарчиг

Хураангуй	i
Товчлолын жагсаалт	ii
Ажлын төлөвлөгөө	iii
Гарчиг	v
Зургийн жагсаалт	vii
Хүснэгтийн жагсаалт	viii
1 Удиртгал	1
1.1 Асуудлын тулгамдсан байдал	1
1.2 Судалгааны асуулт	1
1.3 Зорилго, зорилт	1
1.4 Хамрах хүрээ ба хязгаарлалт	2
1.5 Ажлын бүтэц	2
2 Судалгааны сэдвийн онол, өнөөгийн байдал	3
2.1 Онол, технологийн үндэс	3
2.2 Холбогдох судалгаанууд	5
2.3 Ижил төрлийн системүүдийн судалгаа ба харьцуулалт	6
3 Арга зүй	9
3.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй	9
3.2 Системийн архитектур	11
3.3 Системийн хэрэглэгчид	16
3.4 Системийн шаардлага	18
3.5 Ашигласан технологи	20
3.6 Юзкейс диаграммууд	20
3.7 Дараалал диаграммууд (Sequence Diagrams)	23
3.8 Өгөгдлийн сангийн ерөнхий загвар	26
4 Туршилт, үр дүн	29
4.1 Туршилтын орчин ба аргачлал	29
4.2 Зэрэгцээ хандалтын туршилт	29
4.3 Гүйцэтгэлийн туршилт (Load Test)	30
4.4 Аюулгүй байдлын туршилт	31
4.5 End-to-End (E2E) туршилт	31
4.6 Үр дүнгийн дүгнэлт	32
5 Дүгнэлт	33
5.1 Зорилтуудын хэрэгжилт	33
5.2 Харьцуулсан дүгнэлт	33
5.3 Хийж чадаагүй зүйлс ба цаашдын ажил	34
5.4 Төгсгөл	35

Ном зүй	36
--------------------------	-----------

Зургийн жагсаалт

Зураг 1:	Multi-tenant өгөгдлийн тусгаарлалтын 3 загвар (Chong et al. 2006)	4
Зураг 2:	Pessimistic vs Optimistic concurrency control: write-write contention-ы зан төлөв . . .	4
Зураг 3:	Uber Freight платформ	6
Зураг 4:	Papa Logistics платформ	6
Зураг 5:	Архитектурын 4 хязгаарлалт → тэдгээрийн шийдлийн зураглал	7
Зураг 6:	Системийн ерөнхий бүтэц: 2 клиент апп + нэгдсэн backend	10
Зураг 7:	Системийн үндсэн үйл ажиллагааны урсгал: бүртгэлээс орлого бүртгэл хүртэлх 6 алхамт үе шат	10
Зураг 8:	БааS-д суурилсан архитектурын бүтэц	12
Зураг 9:	Олон давхаргат аюулгүй байдлын архитектур (гаднаас дотогш: сүлжээ → auth → middleware → данн → бизнес логик)	15
Зураг 10:	ePOD OTP-ийн 5 давхаргат хамгаалалт: халдлагын төрөл бүрд харгалзах шийдэл	16
Зураг 11:	Өгөгдлийн урсгалын диаграмм — Level 0 (контекст диаграмм)	18
Зураг 12:	Хүргэгчийн юзкейс диаграмм	21
Зураг 13:	Борлуулагчийн юзкейс диаграмм	22
Зураг 14:	Админы юзкейс диаграмм	23
Зураг 15:	Хүргэгчийн бүртгэл ба баталгаажуулалтын дараалал диаграмм	24
Зураг 16:	Хүргэлтийн даалгавар үүсгэх ба нийтлэх дараалал диаграмм	25
Зураг 17:	Даалгавар авах дараалал диаграмм (зэрэгцээ хандалтын хамгаалалт)	26
Зураг 18:	Системийн өгөгдлийн сангийн үндсэн ERD (10 хүснэгт). Шар delivery_tasks нь системийн төв entity. Тасархай хөх шугам нь courier_kyc_reviewer_id (админ-аар KYC шалгах) холбоо. task_assignments.courier_id, courier_earnings.courier_id, proof_of_delivery.created_by FK-ууд нь бүгд couriers.id руу шилждэг (тодорхой байх үүднээс давтан зураагүй). couriers нь composite FK (id, role)-аар profiles-той нягт холбогдсон тул зөвхөн role='courier' төрөлтэй профайл courier болгон бүртгэгдэнэ.	27
Зураг 19:	k6 load test — 5 endpoint-ийн p50/p95/p99 latency хуваарилалт (100 VU, 30 мин) .	30

Хүснэгтийн жагсаалт

Хүснэгт 1:	Холбогдох судалгааны таксономи: чиглэл → эх сурвалж → энэ ажилд өгсөн хувь нэмэр	5
Хүснэгт 2:	Судалгаа хоорондын архитектурын харьцуулалт	7
Хүснэгт 3:	Архитектурын зөрүү болон дизайн шаардлагын уялдаа	8
Хүснэгт 4:	Серверийн функцүүдийн (RPC) жагсаалт	14
Хүснэгт 5:	RLS бодлогуудын хураангуй	15
Хүснэгт 6:	Системийн технологийн стек	20
Хүснэгт 7:	Зэрэгцээ claim туршилтын үр дүн (N хүргэгч, 10 давталт)	29
Хүснэгт 8:	k6 ачааллын туршилтын endpoint-үүдийн хариу хугацаа (ms)	30
Хүснэгт 9:	RLS бодлогын нэвтрэлтийн тестийн үр дүн	31
Хүснэгт 10:	E2E захиалгын урсгалын баталгаажуулалт	32
Хүснэгт 11:	Системийн чанарын шаардлагуудын туршилтаар баталгаажсан биелэлт	32
Хүснэгт 12:	Зорилтуудын хэрэгжилтийн хураангуй	33
Хүснэгт 13:	Гурван системийн нэгдсэн харьцуулалт	34

1 Удиртгал

1.1 Асуудлын тулгамдсан байдал

Монгол Улсын жижиг, дунд бизнесийн байгууллагууд (ЖДБ) хоорондоо бараа бүтээгдэхүүн захиалах, хүргэх үйл явцаа өдгөө хүртэл гол төлөв утас, мессеж, цаасан бичгээр гардаг [1]. Захиалагч барааг өөрийн биеэр авч ирэх, эсхүл хүргэлтийг тусад нь зохион байгуулах шаардлага нь цаг алдагдуулж, зохион байгуулалтын алдаа үүсгэж, үйл явцыг хянахад төвөгтэй болгож байна.

Дижитал шилжилтийн нөхцөлд B2B харилцааг автоматжуулах эрэлт дэлхий нийтэд эрс нэмэгдэж [2], [3], хүргэлтийн цахим платформууд (≈ 4.5 тэрбум ам.доллар, 2023 он) гол хөдөлгөгч хүч болж байна [4]. Монголын зах зээлд энэ чиглэлийн нэгдсэн шийдэл хараахан төлөвшөөгүй бөгөөд орон нутгийн тоглогчид (Papa Logistics гэх мэт) голдуу dispatcher-т тулгуурласан, нэг талтай загвартай хэвээр байна.

1.2 Судалгааны асуулт

Энэ ажил нь **operator-гүй, multi-tenant B2B хүргэлтийн платформыг одоогийн cloud BaaS технологи дээр найдвартай хэрхэн бүтээх вэ?** гэсэн ерөнхий асуултыг дэвшүүлж, гурван тодорхой дэд асуултыг шалгана:

- **RQ1 — Concurrency:** PostgreSQL-ийн FOR UPDATE SKIP LOCKED механизм нь FCFS claim model-ыг race condition-гүй хэрэгжүүлж чадах уу?
- **RQ2 — Isolation:** Row Level Security (RLS) нь application-layer алдаанаас үл хамааран multi-tenant өгөгдлийн тусгаарлалтыг database түвшинд хангаж чадах уу?
- **RQ3 — Viability:** BaaS-mediated архитектур нь зөвхөн prototype биш, production-д хүрэх системийг хурдан хөгжүүлэх суурь болж чадах уу?

1.3 Зорилго, зорилт

Ажлын **зорилго** нь B2B хүргэлтийн operator-гүй, multi-tenant платформыг Supabase (PostgreSQL + PostgREST + RLS) технологи дээр боловсруулж, гурван дэд асуултыг туршилтаар хариулах явдал юм.

Дэвшүүлсэн **зорилтууд**:

1. **Судалгаа, шинжилгээ:** Монгол Улс дахь B2B хүргэлтийн өнөөгийн байдлыг дүгнэх, Uber Freight, Papa Logistics зэрэг ижил төрлийн платформуудыг харьцуулах, архитектурын зөрүү ба design шаардлагыг тодорхойлох.
2. **Архитектур, загварчлал:** Client layer (React Native/Expo + Next.js), BaaS layer (Supabase), data layer (PostgreSQL + RLS)-ээс тогтсон BaaS-mediated архитектур зохион бүтээх. 11 хүснэгт бүхий schema, ERD, sequence diagram, use case diagram боловсруулах.
3. **Аюулгүй байдал:** RBAC, RLS, JWT authentication, bcrypt ePOD OTP-г нэгтгэсэн олон давхаргат security хэрэгжүүлэх.

4. **Функционал модулиуд:** Хүргэлтийн даалгавар, marketplace, realtime мэдэгдэл, ePOD, орлого, санхүүгийн тайланг бүрэн хэрэгжүүлэх.

1.4 Хамрах хүрээ ба хязгаарлалт

Ажлын хамрах хүрээ нь (i) Монгол Улсын B2B хүргэлтийн orchestration, (ii) дунд зэргийн ачаалал (100 concurrent user хүртэл) бүхий staging орчинд хэрэгжсэн техникийн нотолгоо, (iii) single-region Supabase Pro tier тохиргоогоор хязгаарлагдана. Бодит GPS location tracking, төлбөрийн бодит integration (QPay, SocialPay), ML recommendation нь энэ ажлын хүрээнд биш — цаашдын хөгжүүлэлтийн чиглэл юм (§4.6).

1.5 Ажлын бүтэц

Үлдэж буй хэсэг нь дараах бүтэцтэй. **Бүлэг 2**-т last-mile логистик, two-sided marketplace, multi-tenant архитектур, concurrency control, BaaS-ын онолын суурь болон Uber Freight, Papa Logistics-тэй харьцуулсан судалгааг танилцуулна. **Бүлэг 3**-т системийн архитектур, өгөгдлийн загвар, аюулгүй байдлын бодлого, функционал шаардлагыг дэлгэрэнгүй тайлбарлана. **Бүлэг 4**-т гүйцэтгэсэн туршилтын үр дүн, харьцуулалт болон дүгнэлтийг оруулсан.

2 Судалгааны сэдвийн онол, өнөөгийн байдал

Тус бүлэгт B2B crowdsourced хүргэлтийн системийн онолын суурь, одоогийн платформуудын шинжилгээ, архитектурын шаардлагыг таван багцад нэгтгэв: (i) last-mile логистик, (ii) two-sided marketplace, (iii) multi-tenant isolation, (iv) concurrency control, (v) BaaS архитектур.

2.1 Онол, технологийн үндэс

2.1.1 Last-mile логистик ба crowdsourced загвар

Логистикийн сүлжээний хамгийн өндөр зардалтай үе шат нь last-mile хүргэлт бөгөөд зарим судалгаагаар нийт тээврийн зардлын 13–75%-ийг эзэлдэг [5], [6]. Уламжлалт, өөрийн парктай загвар нь найдвартай боловч өндөр зардалтай, уян хатан бус, өргөтгөхөд хүндрэлтэй [7], [8]. Эдгээр хязгаарлалтыг даван туулахын тулд crowdsourced delivery буюу бие даасан гүйцэтгэгчдийг платформоор холбох загвар бий болжээ [9].

Alnaggar нар (2021) 40 гаруй эрдэм шинжилгээний эх сурвалжийг нэгтгэж crowdsourced загварыг ажилтны ангилал, даалгавар хуваарилалтын механизм, урамшууллын бүтэц гэсэн гурван хэмжээсээр тоймложээ [10]. Mancini ба Gansterer (2022) даалгавар багцлах алгоритмаар тээврийн зардлыг ойролцоогоор 10%-иар бууруулж болохыг харуулсан [11]. Эдгээр ажил нь ихэвчлэн оновчлол (optimization)-д чиглэсэн боловч claim model-ын зөв байдал нь тэрхүү оновчлолын суурь тавцанг бүрдүүлдэг. Тус судалгаа оновчлолд биш, *claim model-ын correctness давхаргыг бэхжүүлэх* шийдэлд чиглэсэн тул optimization давхаргыг future work-т үлдээв.

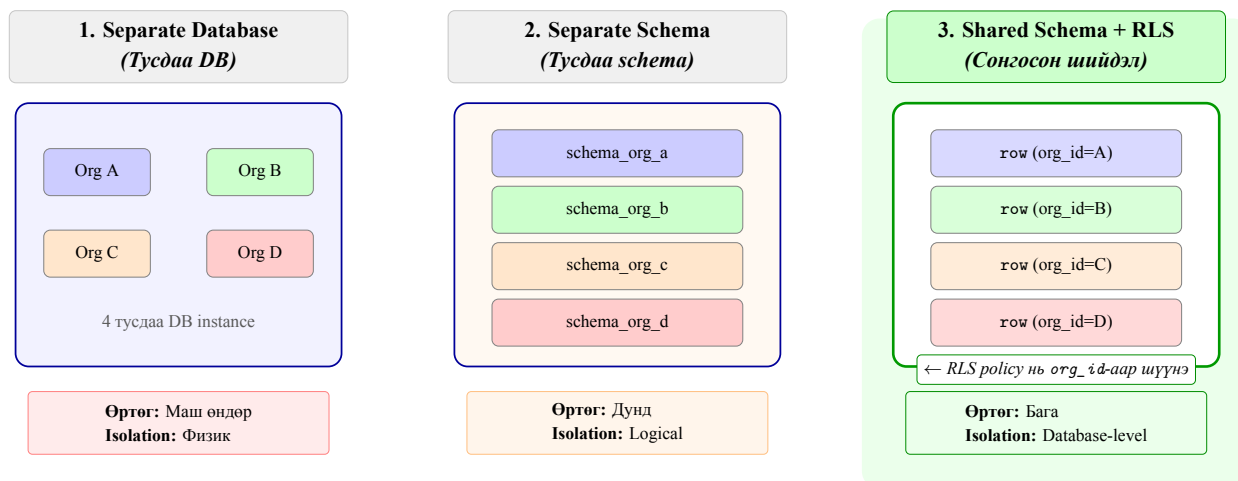
2.1.2 Платформ эдийн засаг ба олон талт зах зээл

Хүргэлтийн платформ нь борлуулагч–хүргэгч гэсэн өөр өөр талуудыг холбодог учир олон талт (two-sided) зах зээлийн ангилалд хамаарна. Rochet ба Tirole (2003) нарын загвараар платформын тэнцвэрт үнийн *бүтцийн хуваарилалт* нь нийт үнийн түвшнээс илүү чухал [12]. Parker нар (2016) болон Evans, Schmalensee (2016) сүлжээний эффект (network effect), матч хийх үр ашиг, cold-start асуудлыг платформын гол хөдөлгөгчүүд гэж тодорхойлсон [13], [14].

Эдгээр онолын үүднээс B2B хүргэлтийн платформын гол саад нь *хүргэгч талын liquidity* болдог. Иймээс cold-start шатанд даалгавар claim хийх хугацааг богиносгох FCFS загвар нь dispatcher-д суурилсан bid хэлбэртэй харьцуулахад давуу талтай. Платформ томрох тусам priority scoring, route optimization гэх мэт алгоритмын давхаргыг үе шаттайгаар нэмэх боломжтой.

2.1.3 Multi-tenant архитектур ба өгөгдлийн тусгаарлалт

Multi-tenant систем дэх өгөгдлийг зохион байгуулахад Chong нар (2006) гурван загварыг тодорхойлсон (Зураг 1): тусдаа database, тусдаа schema, эсвэл хуваалцсан schema дээр row-level isolation [15]. Сүүлчийн хувилбар нь өртөг багатай боловч application-layer алдааны эрсдэл өндөр бөгөөд Bezemer ба Zaidman (2010) нар isolation-ыг *lowest possible layer*-д, ихэвчлэн database түвшинд хэрэгжүүлэхийг зөвлөсөн [16].

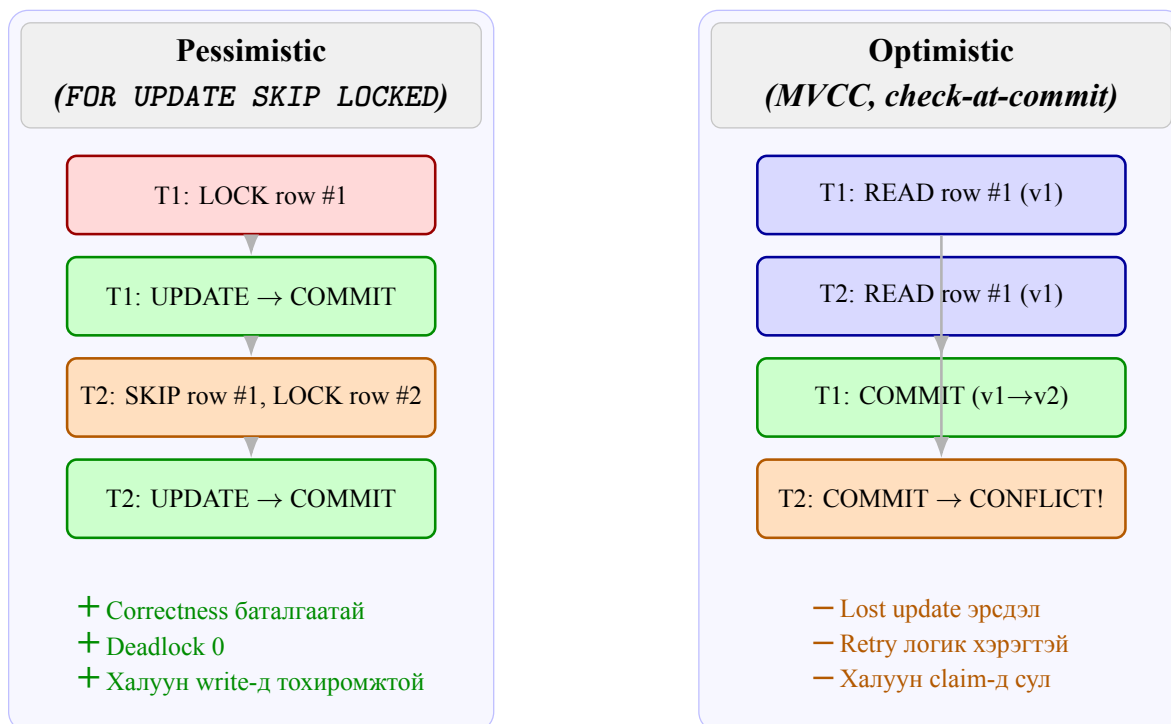


Зураг 1: Multi-tenant өгөгдлийн тусгаарлалтын 3 загвар (Chong et al. 2006)

Энэ зөвлөмжийг хэрэгжүүлэхийн тулд системд PostgreSQL Row Level Security (RLS)-ыг сонгосон. RLS нь хүснэгтийн мөр бүрд USING/WITH CHECK policy хавсаргаж, planner уг policy-г WHERE-д автоматаар нэмдэг [17]. Policy-ийн шалгуур нь Ferraiolo ба Kuhn (1992) нарын RBAC загварын дагуу `auth.jwt()->>'org_id'` claim-аар ажиллана [18]. Ийнхүү RLS нь Chong 2006-ийн таксономи, Beze-mer 2010-ын зөвлөмж, Ferraiolo 1992-ийн RBAC-ийг нэг шийдлээр нэгтгэдэг.

2.1.4 Concurrency control

Bernstein нар (1987) concurrency control-ыг **pessimistic** (транзакцаас өмнө lock авах) ба **optimistic** (commit үед conflict шалгах) гэж ангилсан [19]. Kleppmann (2017) write-heavy workload-д pessimistic хандлага илүү найдвартай гэж дүгнэсэн [20]. Хоёр хандлагын ялгааг Зураг 2-д харуулав.



Зураг 2: Pessimistic vs Optimistic concurrency control: write-write contention-ы зан төлөв

Хүргэгч нэг даалгаврыг зэрэг claim хийх үе нь яг ийм богино, халуун write-write contention тул PostgreSQL-ийн SELECT ... FOR UPDATE SKIP LOCKED construct тохиромжтой: нэг транзакц lock авсан мөрийг өөр транзакц алгасаж дараагийн мөрийг авна [21]. Энэ нь давхар claim-ын магадлалыг 0 болгож, deadlock-ийг бүрмөсөн арилгана.

Kleppmann-ы нэр томъёогоор SKIP LOCKED нь зөвхөн lock primitive биш, харин “database-д суулгасан work queue” хэрэгжүүлэх шийдэл юм [20]. Ингэснээр тусдаа queue service (Redis, Kafka) шаардлагагүйгээр correctness guarantee, performance, audit trail-г нэг эх үүсвэрээс олдог.

2.1.5 BaaS (Backend-as-a-Service) архитектур

Уламжлалт three-tier архитектурт бизнесийн логик нь тусдаа удирддаг application server (Node.js, Spring Boot)-д хэрэгждэг. Харин serverless/BaaS загвар уг давхаргыг платформд даалгаж, pay-per-use model-д шилжүүлсэн [22]. Adzic ба Chatley (2017) уг загвар зардлыг 60–80% хүртэл бууруулж чадах боловч vendor lock-in, cold start, хязгаарлагдмал observability зэрэг trade-off үүсгэдгийг тэмдэглэсэн [23]. Jonas нарын Berkeley-ийн тоймд (2019) stateful үйлчилгээг function-тай хослуулсан нэгдсэн BaaS платформыг cloud programming-ийг хялбаршуулах хамгийн ирээдүйтэй чиглэл гэж дүгнэсэн [24].

Supabase нь PostgreSQL дээр үүссэн BaaS платформ юм. PostgREST нь REST API-г schema-аас автомат үүсгэнэ. RLS нь JWT claim-аар идэвхжинэ. Realtime нь WAL log-оос өөрчлөлтийг авдаг. Иймд бизнесийн логик нь RLS policy, SECURITY DEFINER RPC, триггер, Edge Functions-д тархана. Уг BaaS-mediated загвар нь Adzic 2017-ийн анхааруулсан observability эрсдэлтэй тулгардаг. Тиймээс бизнесийн логикийг database-ээс шууд шалгах туршилтын аргачлалыг Бүлэг 4-т сонгов.

2.2 Холбогдох судалгаанууд

Энэ судалгааны онолын суурийг 5 үндсэн чиглэлд ангилж, Хүснэгт 1-д нэгтгэв. Чиглэл бүрд тус судалгаанд ямар үүрэг гүйцэтгэсэн эх сурвалжийг тусад нь дурдсан.

Хүснэгт 1: Холбогдох судалгааны таксономи: чиглэл → эх сурвалж → энэ ажилд өгсөн хувь нэмэр

Чиглэл	Үндсэн эх сурвалж	Тус судалгаанд өгсөн үүрэг
Crowdsourced хүргэлт	Rougès & Montreuil (2014) [9], Lim нар (2018) [6], Alnagar нар (2021) [10]	Dispatcher-гүй self-service marketplace загварын үндэслэл
Платформ эдийн засаг	Rochet & Tirole (2003) [12], Parker нар (2016) [13], Evans & Schmalensee (2016) [14]	FCFS загварын cold-start давуу талын онолын нотолгоо
Multi-tenant isolation	Chong нар (2006) [15], Bezemer & Zaidman (2010) [16], PostgreSQL RLS docs [17]	Database-level RLS сонголтын таксономи ба зөвлөмж
Access control & auth	Ferraiolo & Kuhn (1992) [18], Provos & Mazières (1999) [25], Jones нар (2015) [26], OWASP (2024) [27]	RBAC, bcrypt ePOD OTP, JWT best-practice-ийн суурь
Concurrency control	Bernstein нар (1987) [19], Kleppmann (2017) [20], PostgreSQL locking docs [21]	Pessimistic locking ба FOR UPDATE SKIP LOCKED сонголтын үндэслэл
BaaS & Serverless	Baldini нар (2017) [22], Adzic & Chatley (2017) [23], Jonas нар (2019) [24]	BaaS-mediated архитектурын сонголт, trade-off-ийн үндэслэл

Энэхүү таксономи нь судалгааны шийдвэр тус бүр — *яагаад FCFS?*, *яагаад RLS?*, *яагаад BaaS?* — онолын эх сурвалжтай нягт уялдсаныг харуулна.

2.3 Ижил төрлийн системүүдийн судалгаа ба харьцуулалт

2.3.1 Uber Freight (олон улсын платформ)



Зураг 3: Uber Freight платформ

Uber Freight нь дижитал брокер, TMS, аналитик, ML шийдлүүдийг нэгтгэсэн хоёр талт платформ юм [28]. Shipper нь FTL/LTL горимоор instant quote-ыг ML-д суурилсан үнийн алгоритмаар авдаг. Carrier нь DOT/МС дугаар, даатгалын шаардлагыг хангаж онбординг хийгдэн, апп/веб порталаар ачаа book хийж, гүйцэтгэлийн баримт (POD) илгээдэг. Платформ нь recommendation system-ээр ажилладаг: carrier-ийн зан төлвийг implicit суралцаж, candidate generation → ranking/booster бүтцээр ачааг оновчтой санал болгодог. Тээвэрлэлтийн дараа бодит цагийн visibility, cargo damage claim зэрэг механизмуудаар гүйцэтгэлийг хянадаг. Нийтдээ Uber Freight нь liquidity, алгоритмын шийдвэр, API-д суурилсан интеграцийг нэгтгэсэн өндөр scalable экосистемийг бүрдүүлдэг [13].

2.3.2 Papa Logistics (Монголын платформ)



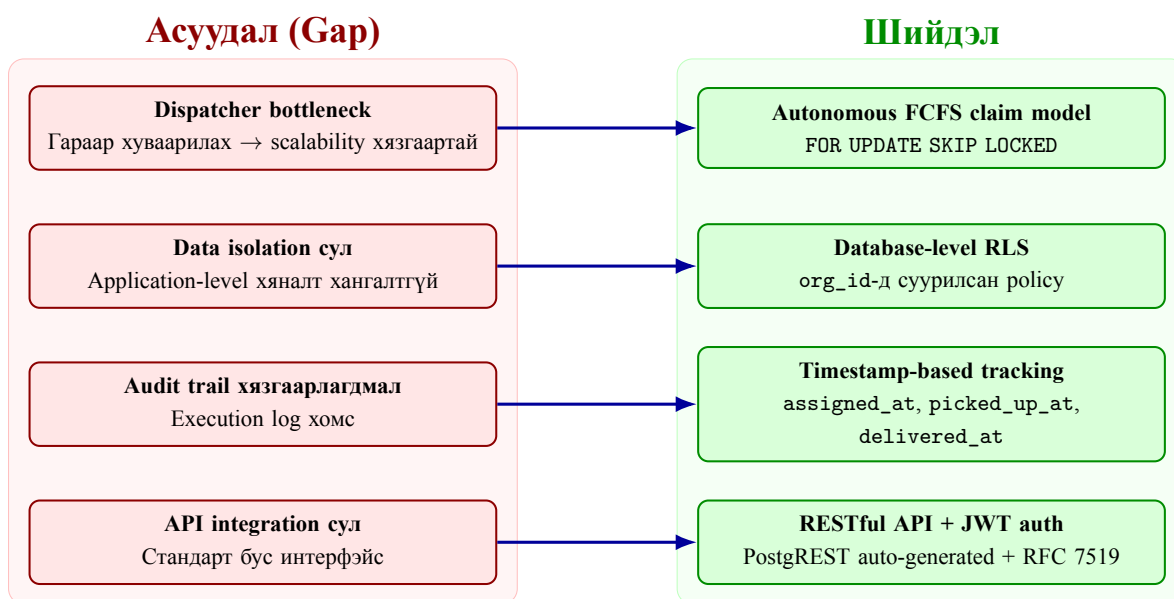
Зураг 4: Papa Logistics платформ

Papa Logistics нь shipper, carrier/жолооч, broker компаниудыг нэг системд холбосон multi-sided логистикийн платформ юм [29]. Transaction lifecycle нь: захиалга үүсгэх → түгээх → үнийн санал

цуглуулах → хэрэглэгч сонгох → location tracking → гүйцэтгэл баталгаажуулах → үнэлгээ. Shipper нь олон үнийн санал харьцуулах, явц хянах, үнэлгээ өгөх боломжтой. Carrier нь тээврийн хэрэгсэл, бүс, үнийн нөхцөлөө тодорхойлж, захиалганд bid илгээдэг. Pricing нь Uber Freight шиг dynamic ML биш, харилцан тохиролцоонд суурилсан. Даатгалын төрөл, нөхөн төлбөрийн журам, хариуцлагын заалтуудын ил тод баримт бичиг хомс нь risk management-ийн тодорхойгүй байдлыг үүсгэж байна. Ерөнхийдөө Papa Logistics нь Монголын дотоод хэрэгцээнд нийцсэн боловч алгоритмын автоматжуулалт, risk management, API integration-ийн хувьд хөгжих боломжтой.

2.3.3 Архитектурын хязгаарлалтууд ба шийдэл

Судалгаанаас 4 үндсэн хязгаарлалт илэрч, тус бүрт тодорхой шийдлийг зураглав (Зураг 5).



Зураг 5: Архитектурын 4 хязгаарлалт → тэдгээрийн шийдлийн зураглал

Эдгээр 4 шийдэл нь ном зүйн онолтой нягт уялдсан: FCFS claim model-ыг Bernstein [19] болон Kleppmann [20]-ны pessimistic locking зарчмаар, RLS-ийг Bezemer [16], Ferraiolo [18]-гийн зөвлөмжөөр, JWT auth-ыг RFC 7519 [26] стандартаар тус тус үндэслэсэн.

2.3.4 Харьцуулалтын хүснэгт ба gap analysis

Хүснэгт 2: Судалгаа хоорондын архитектурын харьцуулалт

Шинж чанар	Uber Freight	Papa Logistics	Судалгааны систем
Task allocation	ML recommendation	Location-based bid	FCFS + pessimistic locking
Pricing	Dynamic ML	Харилцан тохиролцоо	Борлуулагч тогтоодог
Data isolation	Enterprise (тодорхойгүй)	Application-level	Database-level RLS
Audit trail	Execution tracking	Хязгаарлагдмал	Timestamp-based
API integration	API-first, webhook	Сул	RESTful, webhook-ready
Real-time	Location tracking	Location tracking	WebSocket/subscription
Scalability	Horizontal scaling	Dispatcher bottleneck	Event-driven horizontal
Security	Enterprise (proprietary)	Application-level	RLS + role policy
Multi-tenant	Built-in	Тодорхойгүй	Organization-level RLS

Эдгээр шаардлагууд нь судлагдсан системүүдийн хязгаарлалтыг давж, Монгол Улсын B2B логиستيкийн зах зээлд технологийн алхам урагшлуулах боломжийг олгоно.

Хүснэгт 3: Архитектурын зөрүү болон дизайн шаардлагын уялдаа

Тодорхойлогдсон зөрүү	Дизайн шаардлага
Гараар хуваарилалт давамгайл	Marketplace-д суурилсан dynamic task allocation
Multi-tenant isolation сул	RLS-д суурилсан organization-level isolation
Audit trail хязгаарлагдмал	Event sourcing зарчимд тулгуурласан бүрэн audit log
API integration хязгаарлагдмал	API-first, стандарт REST interface
Scalability сул	Horizontal scalable event-driven архитектур
Real-time дамжуулалт сул	Subscription-based push механизм

3 Арга зүй

Тус судалгаа нь crowdsourced хүргэгчдийн сүлжээнд суурилсан B2B хүргэлтийн платформ хөгжүүлэхэд чиглэсэн **практик судалгаа** бөгөөд **загварчлал, хөгжүүлэлт, туршилт** гэсэн гурван үе шаттайгаар Agile аргачлалаар [30] хэрэгжсэн. Iteration бүрд шинэ чадамж нэмж, шалгаж, засах давталттай бүтэцтэй явсан.

Хөгжүүлэлтийн үе шатууд:

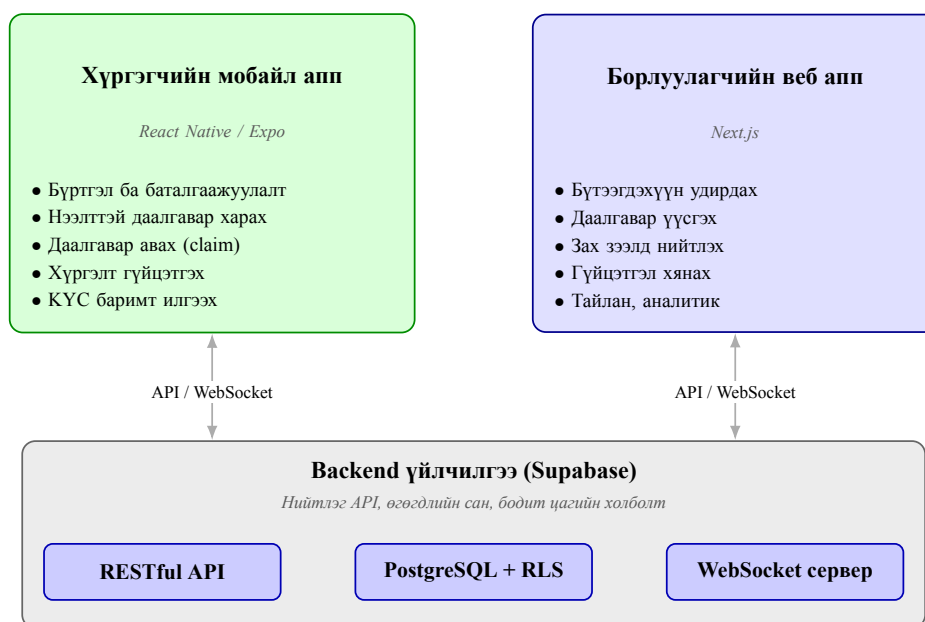
1. **Шаардлагын шинжилгээ:** бизнесийн логик, хэрэглэгчийн хэрэгцээг тодорхойлох
2. **Архитектурын загварчлал:** системийн бүтэц, өгөгдлийн загвар, технологийн стек сонгох
3. **Өгөгдлийн сангийн хөгжүүлэлт:** хүснэгтүүд, Row Level Security (RLS) бодлого, триггер, RPC функцүүд
4. **Backend тохиргоо:** Supabase authentication, file storage, realtime холболт
5. **Frontend хөгжүүлэлт:** мобайл апп (React Native/Expo), веб апп (Next.js)
6. **Нэгтгэлийн туршилт:** бүрэлдэхүүн хэсгүүдийн хоорондын холболт шалгах
7. **Гүйцэтгэлийн туршилт:** зэрэгцээ хандалт, аюулгүй байдлын тестүүд

3.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй

Уг систем нь B2B хүргэлтийн нэгдсэн платформ юм. Байгууллагуудын бараа хүргэлтийг crowdsourced хүргэгчдээр гүйцэтгэнэ. Гол зорилго нь байгууллагад хүргэлтийн тусгай баг байлгах шаардлагагүйгээр, бие даасан хүргэгчдийн marketplace-аар үйлчилгээ авах боломж олгох явдал юм.

3.1.1 Системийн ерөнхий бүтэц

Систем нь хоёр бие даасан клиент аппликейшнээс бүрдэх ба тус бүр нь нэг дундын backend-тэй холбогдоно (Зураг 6).



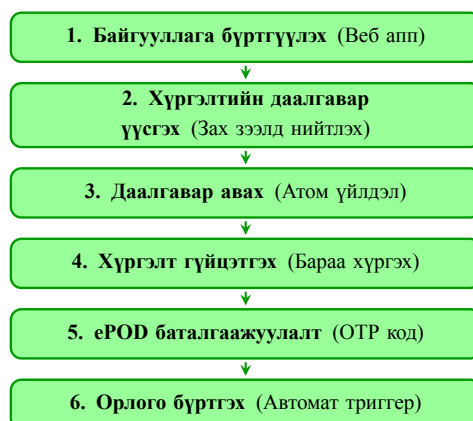
Зураг 6: Системийн ерөнхий бүтэц: 2 клиент апп + нэгдсэн backend

Хүргэгчийн мобайл апп (React Native/Expo) нь хүргэгчдэд зориулсан гар утасны аппликейшн бөгөөд бүртгэл, KYC баталгаажуулалт, нээлттэй даалгавар харах, даалгавар авах, хүргэлт гүйцэтгэх үндсэн үйлдлүүдийг хийнэ.

Борлуулагчийн веб апп (Next.js) нь байгууллагын админд зориулсан веб аппликейшн бөгөөд бүтээгдэхүүнийг удирдах, хүргэлтийн даалгавар үүсгэх, зах зээлд нийтлэх, гүйцэтгэлийг хянах боломжтой.

Хоёр апп нэг дундын Supabase backend-тэй HTTPS (REST API) болон WebSocket протоколоор холбогдоно. Supabase нь PostgreSQL өгөгдлийн сан, authentication, file storage, realtime мэдэгдэл, RPC функцүүдийг нэгтгэсэн Backend-as-a-Service (BaaS) платформ юм [22], [23].

3.1.2 Системийн үндсэн урсгал



Зураг 7: Системийн үндсэн үйл ажиллагааны урсгал: бүртгэлээс орлого бүртгэл хүртэлх 6 алхамт үе шат

3.1.3 Multi-tenant архитектур

Систем нь олон байгууллагад зориулагдсан multi-tenant архитектуртай. Байгууллага бүр өөрийн бүтээгдэхүүн, захиалга, даалгаврын өгөгдлийг бүрэн тусгаарлагдсан хэлбэрээр удирдана. Tenant isolation-ыг PostgreSQL-ийн Row Level Security (RLS) бодлого хангаж, өгөгдлийн сангийн түвшинд баталгаажуулна. Тухайн хэрэглэгчийн org_id-г RPC функцээр тодорхойлж, RLS бодлогод ашиглана.

3.2 Системийн архитектур

3.2.1 Системийн бүрэлдэхүүн хэсгүүдийн тайлбар

Системийн ерөнхий архитектур нь хоёр клиент аппликейшн (хүргэгчийн React Native/Expo апп болон борлуулагчийн Next.js 16 веб апп), нэгдсэн Supabase backend, болон тэдгээрийг холбосон протокол, үйлчилгээнүүдээс бүрдэнэ. Клиентүүд нь Supabase API gateway-ээр (HTTPS/REST болон WebSocket) дамжуулан backend-тэй холбогдоно.

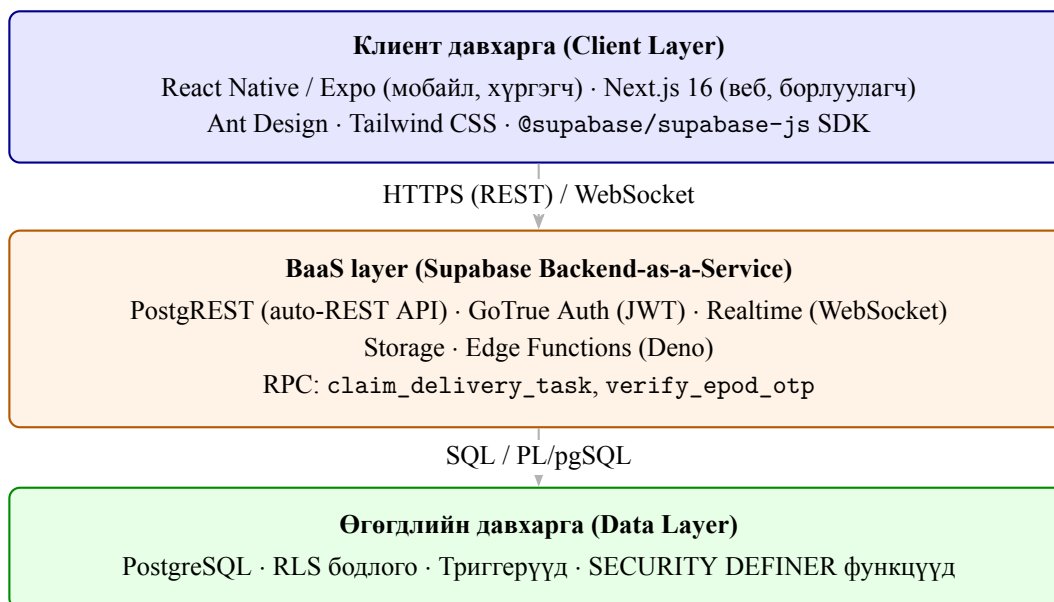
Supabase платформын үндсэн үйлчилгээнүүд нь дараах байдалтай:

- **Auth** — JWT токен үүсгэнэ;
- **PostgreSQL** — RLS бодлоготой өгөгдлийн сан;
- **Storage** — KYC баримт хадгална;
- **Realtime** — WebSocket-д суурилсан мэдэгдэл илгээнэ;
- **RPC функцүүд** — SECURITY DEFINER эрхтэй бизнесийн логик;
- **Edge Functions** — имэйл болон ePOD OTP илгээнэ;
- **Триггерүүд** — төлөв шилжилт, орлогын автомат бүртгэл.

Эдгээр үйлчилгээ хоорондоо JWT token validation, trigger-RPC холболт, Realtime channel-аар уялдана.

3.2.2 BaaS-д суурилсан архитектур (BaaS-mediated architecture)

Уламжлалт *three-tier* загварт бизнесийн логикийг тусдаа application server (Node.js/Express, Spring Boot) дээр байрлуулдаг. Харин тус систем нь тусдаа backend сервер ажиллуулдаггүй. Supabase-ийн BaaS платформ бүх backend үйлчилгээг зуучилдаг. Иймд архитектурыг **BaaS-mediated architecture** [24] гэж нэрлэх нь илүү зөв. API давхарга PostgREST-ээр автоматаар үүсгэгдэнэ. Бизнесийн логик нь RPC, триггер, Edge Functions-д тархсан.



Зураг 8: BaaS-д суурилсан архитектурын бүтэц

Client layer нь хэрэглэгчийн интерфэйс, навигаци, маягтын шалгалт, state management (React Context) зэрэг frontend логикийг хариуцна. Мобайл апп нь React Navigation ашиглан дэлгэцүүдийг удирдаж, веб апп нь Next.js-ийн App Router болон middleware ашиглана. Хоёр клиент хоёулаа @supabase/supabase-js SDK-аар дамжуулан BaaS давхаргатай HTTPS болон WebSocket протоколоор холбогдоно.

BaaS layer нь системийн API давхаргын үүргийг гүйцэтгэдэг Supabase платформ юм. Түүний хамгийн гол бүрэлдэхүүн нь **PostgREST** — PostgreSQL schema-г уншиж, бүх хүснэгт, view, функцийг автоматаар RESTful endpoint болгон хувиргах open source сервер. Ингэснээр клиент нь POST /rest/v1/rpc/claim_delivery_task зэрэг URL-аар шууд өгөгдлийн сангийн функцийг дуудах боломжтой болно. API-гийн дэргэд GoTrue (JWT authentication), Realtime (PostgreSQL WAL-ээс WebSocket мэдэгдэл), Storage (файл), Edge Functions (Deno runtime дээр ажилладаг имэйл илгээх гэх мэт гадаад integration) зэрэг туслах үйлчилгээнүүд ажиллана.

Data layer нь PostgreSQL өгөгдлийн сан, Row Level Security (RLS), триггерүүд, SECURITY DEFINER RPC функцүүдийг багтаана. Бизнесийн логикийн дийлэнх хэсэг — permission check, state machine, зэрэгцээ хандалтын FOR UPDATE SKIP LOCKED хамгаалалт — нь энэ давхаргад PL/pgSQL кодоор хэрэгжсэн. Энэ нь BaaS-mediated архитектурын өвөрмөц шинж бөгөөд клиент талын логигоос хамааралгүй аюулгүй ажиллана.

3.2.3 Schema drift-ээс хамгаалах стратеги

BaaS-mediated архитектурын гол сөрөг тал нь клиент аппликейшн PostgREST-ээр дамжуулан schema-тай шууд холбогддог явдал юм. Хүснэгтийн баганын нэр, төрөл өөрчлөгдөх бүрт клиент код зохицож шинэчлэгдэх шаардлагатай. Уг хамаарлыг *schema drift* гэдэг — database-ийн хэлбэр ба клиентийн таамагласан хэлбэрийн хоорондын зөрүү [31]. Гараар удирдвал runtime-д column ``...'' does not exist алдаа гарч, production-д хагарал үүсгэх эрсдэлтэй.

Уг эрсдэлийг эхнээс нь хаахын тулд дараах **schema drift prevention strategy**-г төсөлд хэрэгжүүлэв:

1. Автомат TypeScript type үүсгэлт. Supabase CLI-ийн supabase gen types typescript

--linked команд нь production-той холбогдсон (linked) төслийн schema-г уншиж, бүх хүснэгт, view, enum, RPC функцийг signature-ийг нэгтгэсэн types/database.ts файлыг үүсгэнэ. Үүнд баганын нэр, төрөл, NOT NULL шинжийг compile-time-д илэрхийлэх Database[``public''] [``Tables''] [``orders''] [``Row''] гэх мэт type-ууд багтана.

2. **CI pipeline integration.** Уг командыг GitHub Actions workflow-д шат болгон оруулсан. Pull request нээгдэх бүрт CI нь linked төслөөс шинэ database.ts үүсгэж, commit-логдсон хувилбартай харьцуулна. Зөрүү илэрвэл build унаж, хөгжүүлэгчийг schema migration-ы дараа type-ийг дахин үүсгэж commit-лохыг шаарддаг.
3. **Compile-time contract enforcement.** Клиент код нь үүсгэгдсэн типийг импортлоод Supabase client-ийг createClient<Database>(url, key) гэх хэлбэрээр параметерждэг. Ингэснээр supabase.from(``orders').select(``customer_id, total') хэлбэрийн дуудлага бүр статик шалгалтад орж, байхгүй багана эсвэл буруу төрөл дуудахад TypeScript compiler-ээр runtime-аас өмнө илэрнэ.
4. **Migration-д суурилсан schema удирдлага.** Өгөгдлийн сангийн бүтцийг зөвхөн supabase/migrations/ хавтасны SQL migration файлаар өөрчлөх дүрэм тогтоосон. Supabase Studio эсвэл psql-ээр шууд гараар өөрчлөхийг хориглож, ингэснээр schema-ийн түүх git-ээр бүрэн хянагддаг болгосон.

Уг стратегийн үр дүнд client-server contract нь compile-time-д баталгаажиг, schema өөрчлөлт runtime-д биш build-time-д илрэнэ. Гэсэн хэдий ч coupling-ийн үндсэн асуудлыг бүрэн шийдэхгүй тул BFF (Backend-for-Frontend) давхарга нэмэх нь цаашдын ажлын чухал чиглэл юм.

3.2.4 Authentication архитектур

Систем нь хоёр ялгаатай authentication арга ашиглана — веб аппд серверийн талын (SSR) authentication, мобайл аппд клиент талын authentication.

Борлуулагчийн веб апп (Next.js SSR)

1. Хэрэглэгч имэйл, нууц үгээр нэвтэрнэ
2. Supabase Auth сесс үүсгэж, JWT токен олгоно
3. Токен серверийн талын cookie-д хадгалагдана
4. Next.js middleware нь хүсэлт бүрд сессийг шалгаж, токен шинэчилнэ
5. Хамгаалагдсан замууд (/dashboard, /tasks, /orders г.м.) нэвтрээгүй бол нэвтрэх хуудас руу чиглүүлнэ
6. Борлуулагчийн layout нь хэрэглэгчийн эрхийн түвшинг (role) шалгаж, зөвхөн зөвшөөрөгдсөн эрхтэй хэрэглэгчийг нэвтрүүлнэ

Хүргэгчийн мобайл апп (React Native)

1. Хэрэглэгч имэйл, нууц үгээр нэвтэрнэ (Supabase Auth)
2. Сесс AsyncStorage-д хадгалагдана (оффлайн хандалтын дэмжлэг)

3. CourierAuthContext нь authentication төлвийг нэгдсэн байдлаар удирдана
4. Апп эхлэхэд restoreSession() ажиллаж, хүчинтэй сессийг сэргээнэ
5. Хүргэгчийн эрхийн түвшин (courier), батламжийн төлөв (pending/approved/blocked) шалгагдана
6. Батлагдаагүй бол KYC шаардлагатай эсвэл хүлээж буй дэлгэц харуулна
7. Төлөв “pending” бол 60 секунд тутам автоматаар шинэчилнэ (админ батлахад шууд шинэчлэгдэнэ)

3.2.5 RPC (Remote Procedure Call) архитектур

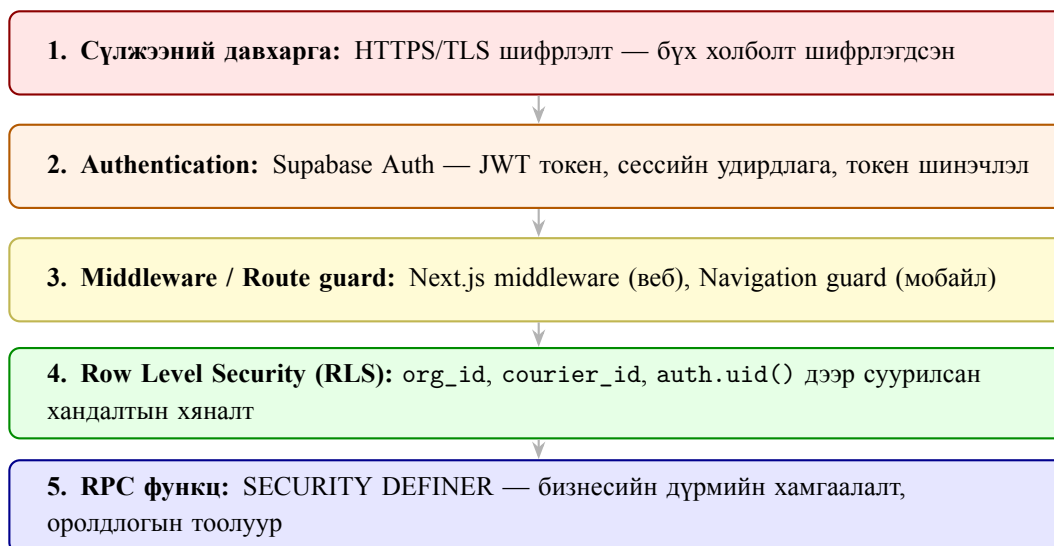
Бизнесийн логикийн гол хэсэг нь PostgreSQL-ийн RPC (Remote Procedure Call) функцүүдэд хэрэгжсэн. Эдгээр функц нь SECURITY DEFINER эрхтэй ажиллаж, RLS бодлогоос үл хамааран бизнесийн дүрмийг нэгдмэл байдлаар хэрэгжүүлнэ.

Хүснэгт 4: Серверийн функцүүдийн (RPC) жагсаалт

Функц	Эрх	Тайлбар
claim_delivery_task	SECURITY DEFINER	Хүргэгч даалгавар авах. FOR UPDATE SKIP LOCKED ашиглан зэрэгцээ хандалтаас хамгаалсан. KYC шалгалт, зөвхөн нэг идэвхтэй даалгавартай байх нөхцөл шалгана.
update_task_status	SECURITY DEFINER	Төлвийн шилжилт: assigned → picked_up → delivered. Цагийн тэмдэг автоматаар бүртгэгдэнэ.
get_available_tasks	SECURITY DEFINER	Нээлттэй даалгаврын жагсаалт. RLS-ийн рекурсив асуудлыг шийдэж, байршлыг хамт буцаана.
request_epod_otp	SECURITY DEFINER	6 оронтой OTP үүсгэнэ. Bcrypt хэш, 10 минутын хугацаатай. Энгийн текст нэг удаа буцаана.
verify_epod_otp	SECURITY DEFINER	OTP шалгах. Bcrypt харьцуулалт, 10 мин хугацаа, 5 оролдлогын хязгаар. Амжилттай бол delivered → completed.
is_approved_courier	SECURITY DEFINER	Батлагдсан хүргэгч эсэхийг шалгана. RLS бодлогод ашиглагдана.

3.2.6 Аюулгүй байдлын архитектур

Системийн аюулгүй байдал нь олон давхаргат хамгаалалтаар хэрэгжсэн:



Зураг 9: Олон давхаргат аюулгүй байдлын архитектур (гаднаас дотогш: сүлжээ → auth → middleware → данн → бизнес логик)

Row Level Security (RLS) бодлогууд

Системийн гол хүснэгтүүдэд дараах RLS бодлогууд тохируулагдсан:

Хүснэгт 5: RLS бодлогуудын хураангуй

Хүснэгт	Бодлого	Нөхцөл
profiles	Өөрийн профайл	<code>auth.uid() = id</code> — зөвхөн өөрийн мэдээлэл унших, засах
delivery_tasks	Хүргэгчийн харах	<code>is_approved_courier()</code> бөгөөд <code>status = 'published'</code> эсвэл <code>courier_id = auth.uid()</code>
delivery_tasks	Борлуулагчийн удирдлага	<code>org_id = get_user_org_id()</code> — зөвхөн өөрийн байгууллагын даалгавар
courier_earnings	Хүргэгчийн орлого	<code>courier_id = auth.uid()</code> — зөвхөн өөрийн орлого
courier_kyc	Хүргэгчийн баримт	Хүргэгч өөрийн баримт, админ бүгд
delivery_epod_otps	ePOD кодууд	Борлуулагч зөвхөн өөрийн байгууллагын даалгаврын OTP

ePOD OTP аюулгүй байдал

OTP кодын аюулгүй байдлыг 5 түвшний хамгаалалтаар хангасан (Зураг 10).



Зураг 10: ePOD OTP-ийн 5 давхаргат хамгаалалт: халдлагын төрөл бүрд харгалзах шийдэл

3.3 Системийн хэрэглэгчид

Платформ нь олон талт зах зээлийн зарчимд суурилсан бөгөөд дараах гурван үндсэн оролцогчийн харилцан үйлчлэлд тулгуурлана.

3.3.1 Байгууллага (Борлуулагч)

Байгууллага нь хүргэлтийн хэрэгцээ шаардлага үүсгэдэг **эрэлтийн тал** юм. Платформ нь дараах төрлийн байгууллагуудын хүргэлтийг дэмждэг:

- **Ресторан:** хоолны хүргэлт
- **Дэлгүүр:** жижиглэн худалдааны бүтээгдэхүүн
- **Эмийн сан:** эм, эрүүл мэндийн бүтээгдэхүүн
- **Агуулах:** том хэмжээний ачаа хүргэлт

Байгууллагын хэрэглэгчийн үндсэн чадамж:

1. Веб аппаар байгууллагын бүртгэл үүсгэх, мэдээлэл оруулах
2. Бүтээгдэхүүний каталог удирдах (нэмэх, засах, устгах)
3. Хэрэглэгчээс ирсэн захиалга хүлээн авах, боловсруулах
4. Хүргэлтийн даалгавар үүсгэх, хүргэлтийн хураамж тогтоох
5. Даалгаврыг зах зээлд нийтлэх (нийтлэгдсэн төлөвт шилжүүлэх)
6. Хүргэлтийн явц хянах, гүйцэтгэлийг баталгаажуулах

Борлуулагч нь веб аппликейшнд дараах хуудсуудыг ашиглана:

- **Хянах самбар (Dashboard):** гол үзүүлэлтүүд (KPI) — нийт даалгавар, идэвхтэй даалгавар, дууссан даалгавар, орлогын мэдээлэл, өнөөдөр ба өчигдрийн харьцуулалт, бодит цагийн холболтын төлөв
- **Бүтээгдэхүүн:** бүтээгдэхүүний каталог удирдлага (нэмэх, засах, устгах, боломжтой/боломжгүй хугацаа тохируулах)
- **Даалгавар:** хүргэлтийн даалгавар үүсгэх, жагсаалт (төлвөөр шүүх, хуудаслалт), дэлгэрэнгүй, олноор нийтлэх (bulk publish), төлвийн шинэчлэл
- **Захиалга:** захиалгын удирдлага — гурван табтай (“Үйлдэл шаардлагатай” — төлбөр хийгдсэн захиалгууд, “Идэвхтэй” — бэлтгэж буй/бэлэн, “Түүх” — дууссан/цуцлагдсан). Захиалга бүрийн дэлгэрэнгүй мэдээлэл, бараа задаргаа, төлвийн шинэчлэл
- **Санхүүгийн тайлан (Financials):** өнөөдрийн орлого, өчигдрийн орлого, нийт орлого, хүргэлт бүрийн орлогын задаргаа
- **Аналитик (Analytics):** даалгаврын гүйцэтгэлийн шинжилгээ, орлогын чиг хандлага (Recharts график сантай)
- **Тохиргоо:** байгууллагын мэдээлэл засах, ажлын цагийн тохиргоо

3.3.2 Хүргэгч

Хүргэгч нь **нийлүүлэлтийн тал** бөгөөд бие даасан гүйцэтгэгч юм. Хүргэгч нь гар утасны аппликейшн ашиглан системтэй харилцана.

Хүргэгчийн бүртгэлийн болон ажиллагааны үе шатууд:

1. **Бүртгүүлэх:** мобайл аппаар имэйл, нууц үгээр бүртгүүлнэ (эрхийн түвшин автоматаар хүргэгч болж тохируулагдана)
2. **Баталгаажуулалтын баримт илгээх:** иргэний үнэмлэхний зураг (урд/ар), тээврийн хэрэгслийн бүртгэл, хувийн зураг
3. **Админ батламж хүлээх:** баримт илгээсний дараа “илгээсэн” төлөвт шилжиж, админ шалгана
4. **Батлагдсан:** нээлттэй даалгавар харах, авах эрхтэй болно
5. **Даалгавар авах:** зах зээлээс даалгавар сонгон авна
6. **Хүргэлт гүйцэтгэх:** бараа авах → хэрэглэгчид хүргэх
7. **Орлого хянах:** гүйцэтгэсэн хүргэлтийн орлогыг харна

Хүргэгч мобайл аппын үндсэн хэсгүүд:

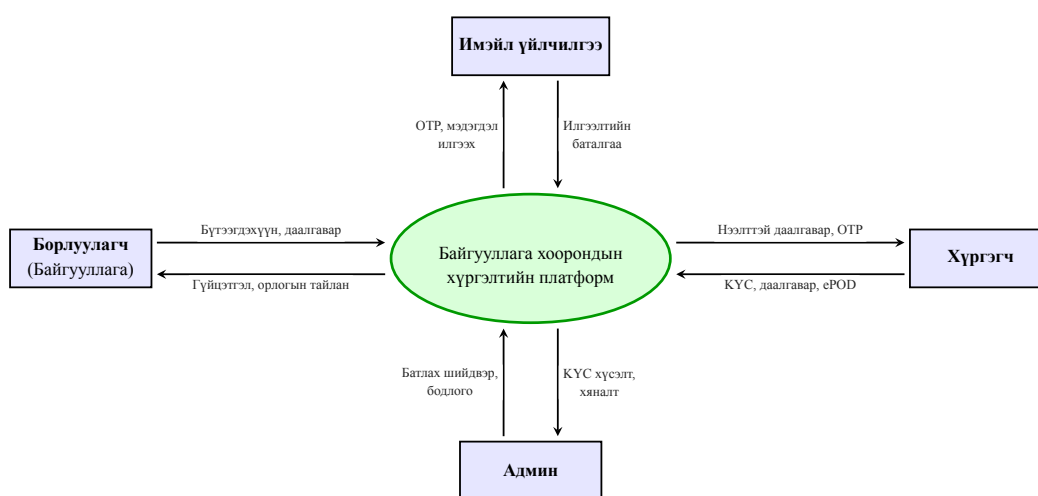
- **Нээлттэй даалгавар:** зах зээлийн нээлттэй даалгаврын жагсаалт
- **Нүүр:** хүргэгчийн хянах самбар, тойм мэдээлэл
- **Захиалга:** идэвхтэй болон хуваарилагдсан захиалгууд

- **Орлого:** хүргэлтийн орлогын түүх
- **Профайл:** хувийн мэдээлэл засах

3.3.3 Админ

Админ нь системийн ерөнхий удирдлага хийх давуу эрхтэй хэрэглэгч юм.

1. Хүргэгчийн баталгаажуулалтын баримтыг шалгах, батлах эсвэл татгалзах
2. Хэрэглэгчийн эрхийн түвшин өөрчлөх
3. Хүргэгчийн эрх хаах (хоригдсон төлөвт шилжүүлэх)
4. Системийн ерөнхий хяналт хийх



Зураг 11: Өгөгдлийн урсгалын диаграмм — Level 0 (контекст диаграмм)

3.4 Системийн шаардлага

Энэ хэсэгт системийн шаардлагуудыг модуль бүрээр задалж тодорхойлно.

3.4.1 Нэвтрэх ба authentication систем

Систем нь Supabase Auth-ийн дэд бүтэц дээр суурилсан. Хэрэглэгч бүртгүүлэх үед апп-аас хамаарч эрх автомат тохируулагдана (хүргэгчийн апп — хүргэгч, веб — борлуулагч); түвшин өөрчлөх зөвхөн админд боломжтой. Техникийн хэрэгжүүлэлтийг (JWT, сесс, триггер) 3.2.4-р хэсэгт тодорхойлсон.

Хүргэгчийн KYC процесс нь **Хүлээгдэж буй** → **Баримт илгээсэн** → **Батлагдсан** (эсвэл **Хоригдсон**) гэсэн төлвийн шилжилтээр явагдана. Илгээх баримт: иргэний үнэмлэх (урд/ар), тээврийн хэрэгслийн бүртгэл, хувийн зураг, тээврийн хэрэгслийн мэдээлэл (Supabase Storage-д RLS-ээр хамгаалагдан хадгалагдана).

3.4.2 Байгууллагын удирдлага

Олон байгууллагын өгөгдлийн тусгаарлалт

Олон байгууллага нэг системд зэрэг ажиллах боловч байгууллага бүрийн өгөгдөл бүрэн тусгаарлагдсан байна (тусгаарлалтын RLS бодлогуудыг 3.2.6-р хэсэгт тодорхойлсон).

- Байгууллагын хүснэгт: дугаар, нэр, төрөл (ресторан, дэлгүүр, эмийн сан, агуулах), үүсгэсэн огноо
- Хэрэглэгч-байгууллагын холбоос: профайл дахь байгууллагын дугаар (хүргэгч, хэрэглэгчид хамаарахгүй)

Байгууллагын бүртгэлийн урсгал (Onboarding)

Шинэ борлуулагч бүртгүүлэх үед дараах үе шатуудыг дамжина:

1. Имэйл, нууц үгээр хэрэглэгчийн бүртгэл үүсгэх
2. Байгууллагын мэдээлэл оруулах — нэр, төрөл (ресторан, дэлгүүр, эмийн сан, агуулах), хаяг, холбоо барих мэдээлэл
3. Амжилттай бүртгэл дууссаны дараа хянах самбар руу шилжинэ

Нууц үг мартсан тохиолдолд имэйлээр нууц үг сэргээх (forgot password / reset) боломж хэрэгжсэн. Supabase-ийн сэргээлтийн линк илгээх механизмыг ашиглав.

Бүтээгдэхүүний каталог удирдлага

Байгууллага өөрийн бүтээгдэхүүнүүдийг нэмэх, засах, устгах, харах үйлдлээр удирдана. Бүтээгдэхүүн бүр тухайн байгууллагын дугаартай холбоотой бөгөөд Row Level Security (RLS)-ээр тусгаарлагдсан.

3.4.3 Захиалгын удирдлагын систем

Захиалгын амьдралын мөчлөг нь **Төлбөр хүлээгдэж буй** → **Төлбөр хийгдсэн** → **Бэлтгэж буй** → **Хүргэлтэд бэлэн** → **Бараа авсан** → **Хүргэгдсэн** шилжилтээр явагдана; тодорхой төлвүүдээс **Цуцлагдсан** төлөвт шилжих боломжтой. Төлбөрийн баталгаажуулалт нь ӨС-гийн триггерээр нэгдмэл байдлаар хийгдэж, төлбөрийг гараар өөрчлөх боломжгүй.

Борлуулагчийн веб апп нь захиалгыг гурван табтай бүтэцэд харуулна: “Үйлдэл шаардлагатай” (төлбөр хийгдсэн боловч бэлтгэгдээгүй), “Идэвхтэй” (бэлтгэж буй/бэлэн), “Түүх” (хүргэгдсэн/цуцлагдсан). Захиалга бүрийг дэлгэрэнгүй харах, баталгаажуулалтын цонхоор төлөв шилжүүлэх боломжтой.

3.4.4 Хүргэлтийн даалгаврын систем

Энэ модуль нь системийн үндсэн шинэлэг шийдэл болох зах зээлд суурилсан хүргэлтийн хуваарилалтыг хэрэгжүүлнэ. Байгууллага хүргэлтийн даалгавар үүсгэхдээ авах/хүргэх байршил (хаяг, координат), хүргэлтийн хураамж, тайлбарыг оруулна; даалгаврыг “нийтлэгдсэн” төлөвт шилжүүлснээр зах зээлд бүх батлагдсан хүргэгчид харагдана.

Даалгаврын төлвийн шилжилт: **Ноорог** → **Нийтлэгдсэн** → **Хуваарилагдсан** → **Бараа авсан** → **Хүргэгдсэн** (ePOD хүлээж буй) → **Дууссан** (орлого бүртгэгдэнэ); тусдаа **Цуцлагдсан** төлөв байна. Олон хүргэгч нэг даалгаврыг зэрэг авахыг оролдоход `claim_delivery_task` функц 3.2.5-р

хэсэгт тайлбарласан атомик түгжээний механизмаар зөвхөн нэгийг нь амжилттай болгоно. Төлөв өөрчлөгдөх бүрт холбогдсон талуудад Supabase Realtime мэдэгдэл очно.

3.4.5 Цахим хүргэлтийн баталгаа (ePOD) ба орлогын систем

ePOD нь хүргэлт амжилттай гүйцэтгэгдсэнийг нотлох OTP суурилсан дижитал баталгаажуулалт юм. Ажиллагааны урсгал:

1. Хүргэгч “хүргэгдсэн” төлөвт шилжүүлнэ
2. Систем 6 оронтой OTP үүсгэж хүлээн авагчид илгээнэ
3. Хүлээн авагч кодыг хүргэгчид хэлж, хүргэгч мобайл аппд оруулна
4. Амжилттай бол даалгавар “дууссан” төлөвт шилжиж, хүргэгчийн орлого автоматаар `courier_earnings` хүснэгтэд бүртгэгдэнэ

Техникийн хэрэгжүүлэлтийн дэлгэрэнгүй мэдээллийг (RPC функцүүд, аюулгүй байдлын механизм) 3.2.5-р хэсэгт тодорхойлсон болно.

Хүргэгч мобайл аппын “Орлого” хэсгээр нийт орлого, хүргэлт бүрийн задаргаа, дууссан хүргэлтийн түүхийг харна.

3.5 Ашигласан технологи

Системийн хөгжүүлэлтэд ашигласан үндсэн технологи, хэрэгслүүдийг дараах хүснэгтэд нэгтгэн харууллаа.

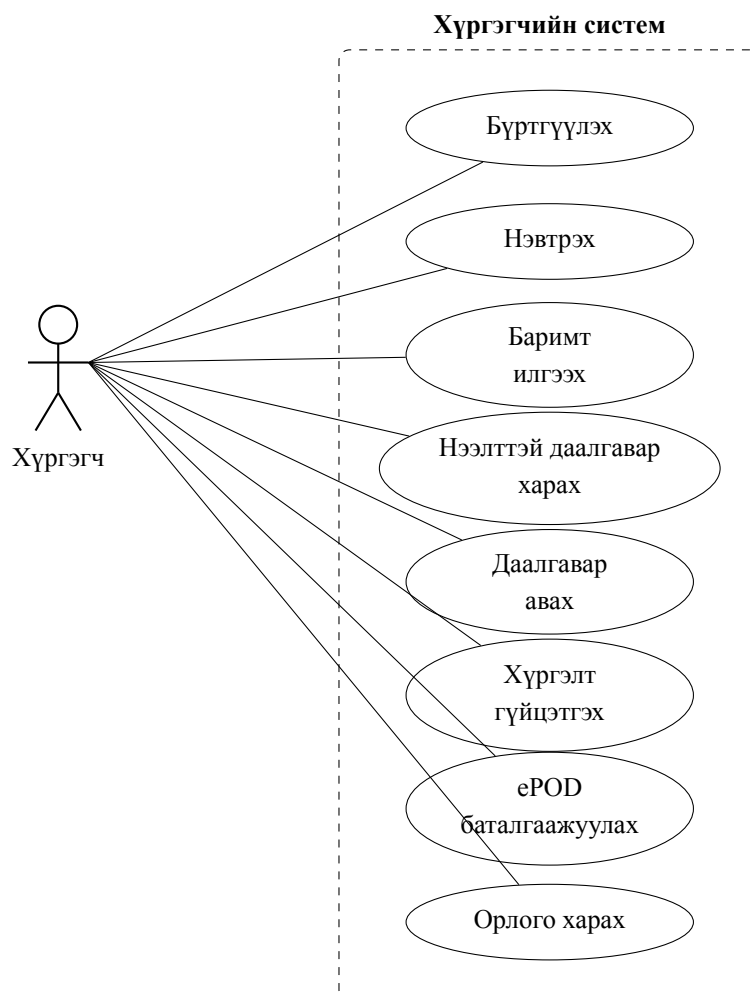
Хүснэгт 6: Системийн технологийн стек

Хэсэг	Технологи	Зориулалт / Хувилбар
Хүргэгчийн мобайл апп	React Native [32] + Expo, TypeScript	Гар утасны frontend framework (Expo 54, TS 5.9). React Navigation, React Hook Form + Zod, AsyncStorage, Reanimated ашиглана.
Борлуулагчийн веб апп	Next.js, React, TypeScript	SSR дэмжсэн веб апп (Next.js 16.1, React 19.2). Ant Design + Tailwind CSS-ээр UI, middleware-ээр сесс, замын хамгаалалт.
Backend	Supabase	PostgreSQL 15+, authentication (JWT), file storage, RPC функц, триггер, RLS.
Хөгжүүлэлтийн хэрэгсэл	VS Code, Git, Node.js, Supabase CLI, Expo CLI	Код бичих, хувилбарын удирдлага, багц, шилжүүлэлт, мобайл бүтээлт.

3.6 Юзкейс диаграммууд

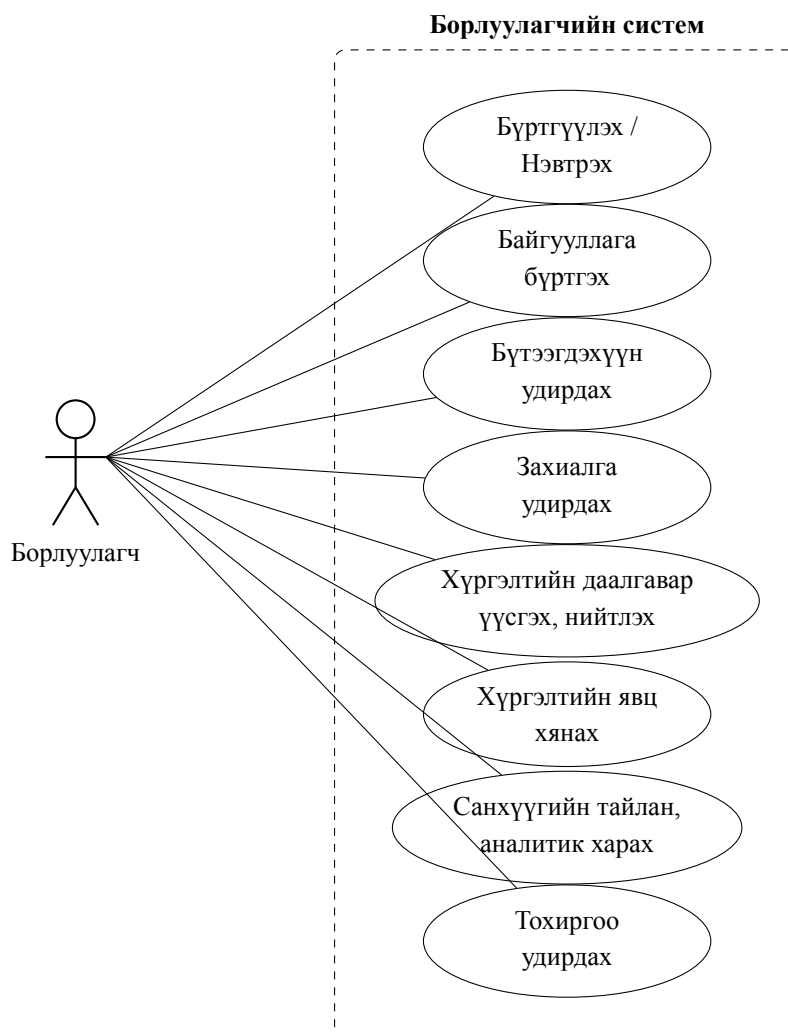
Энэ хэсэгт системийн үндсэн оролцогч тус бүрийн хэрэглээний тохиолдлыг (use case) диаграммаар дүрсэлнэ.

3.6.1 Хүргэгчийн юзкейс диаграмм



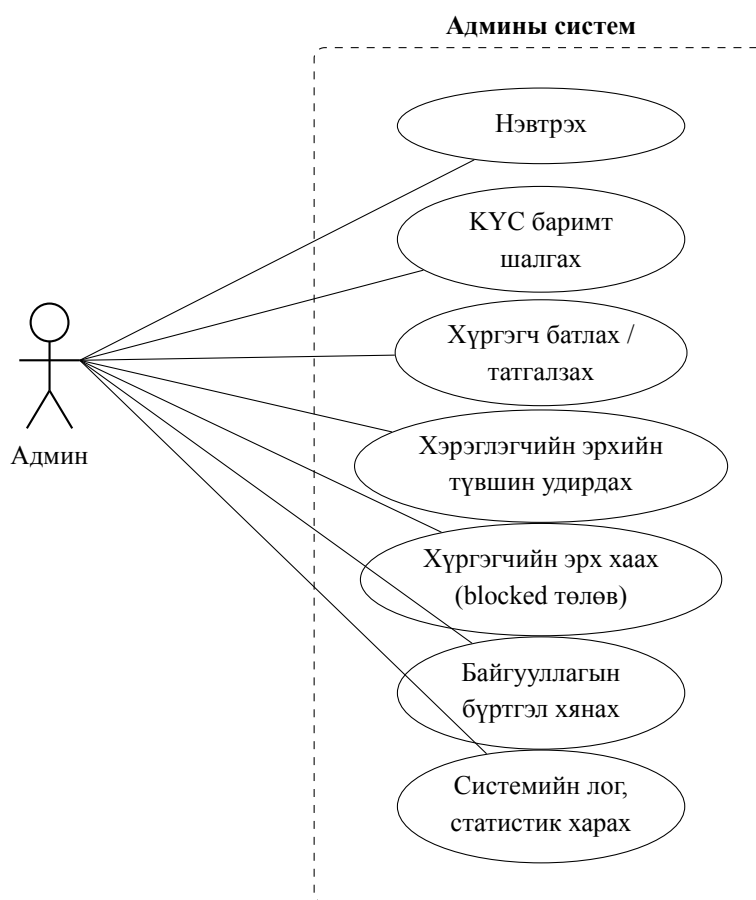
Зураг 12: Хүргэгчийн юзкейс диаграмм

3.6.2 Борлуулагчийн юзкейс диаграмм



Зураг 13: Борлуулагчийн юзкейс диаграмм

3.6.3 Админы юзкейс диаграмм



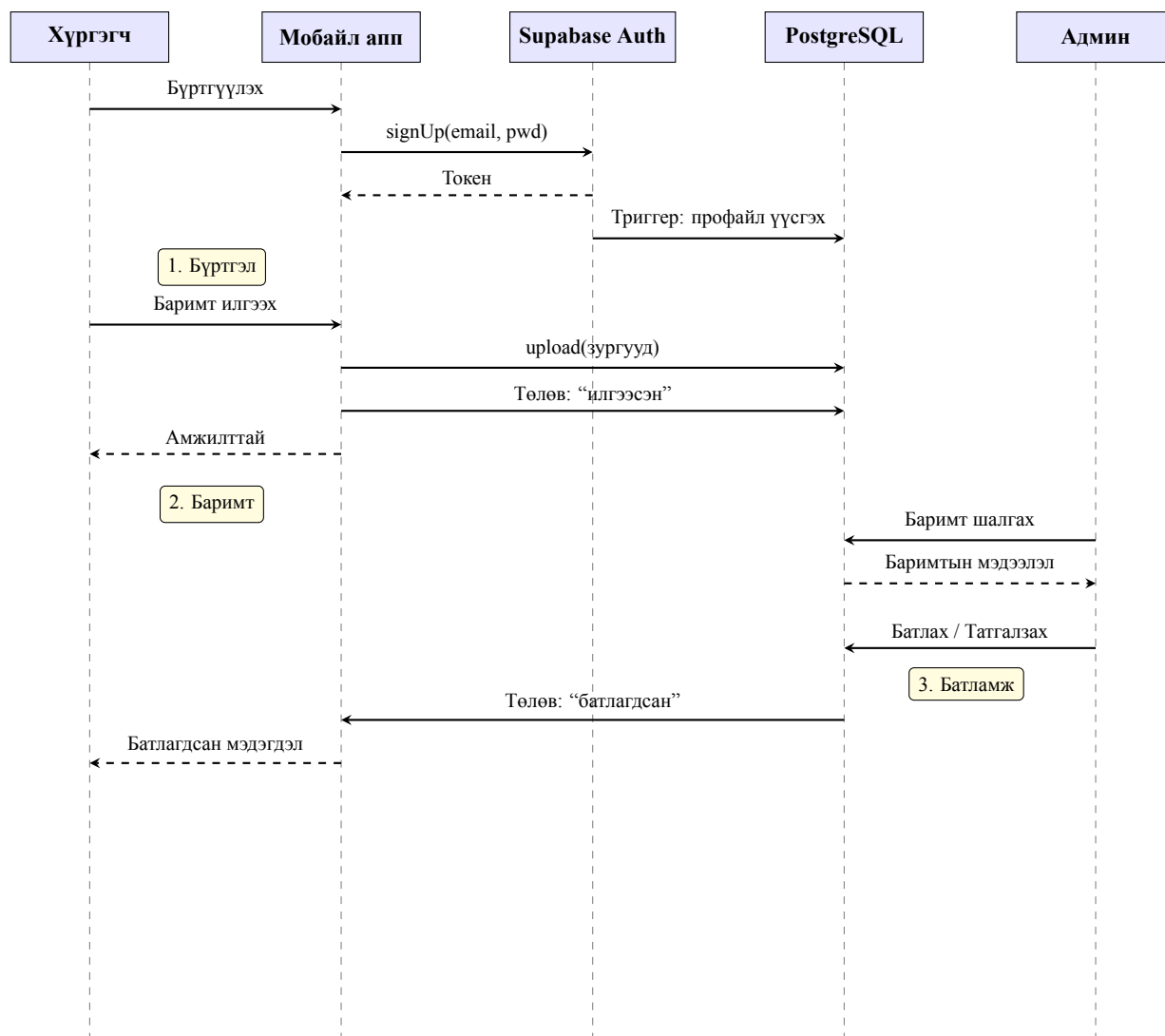
Зураг 14: Админы юзкейс диаграмм

Админы үндсэн үүрэг нь платформын найдвартай ажиллагаа, KYC процессийн чанар, хэрэглэгчийн эрхийн удирдлагыг хангах юм. Админ нь бусад хэрэглэгчээс ялгаатай нь ****админ эрхтэй**** (role = 'admin') тусгай хэрэглэгч бөгөөд RLS bypass шинжтэй SECURITY DEFINER функцээр KYC баримтуудад бүрэн хандана.

3.7 Дараалал диаграммууд (Sequence Diagrams)

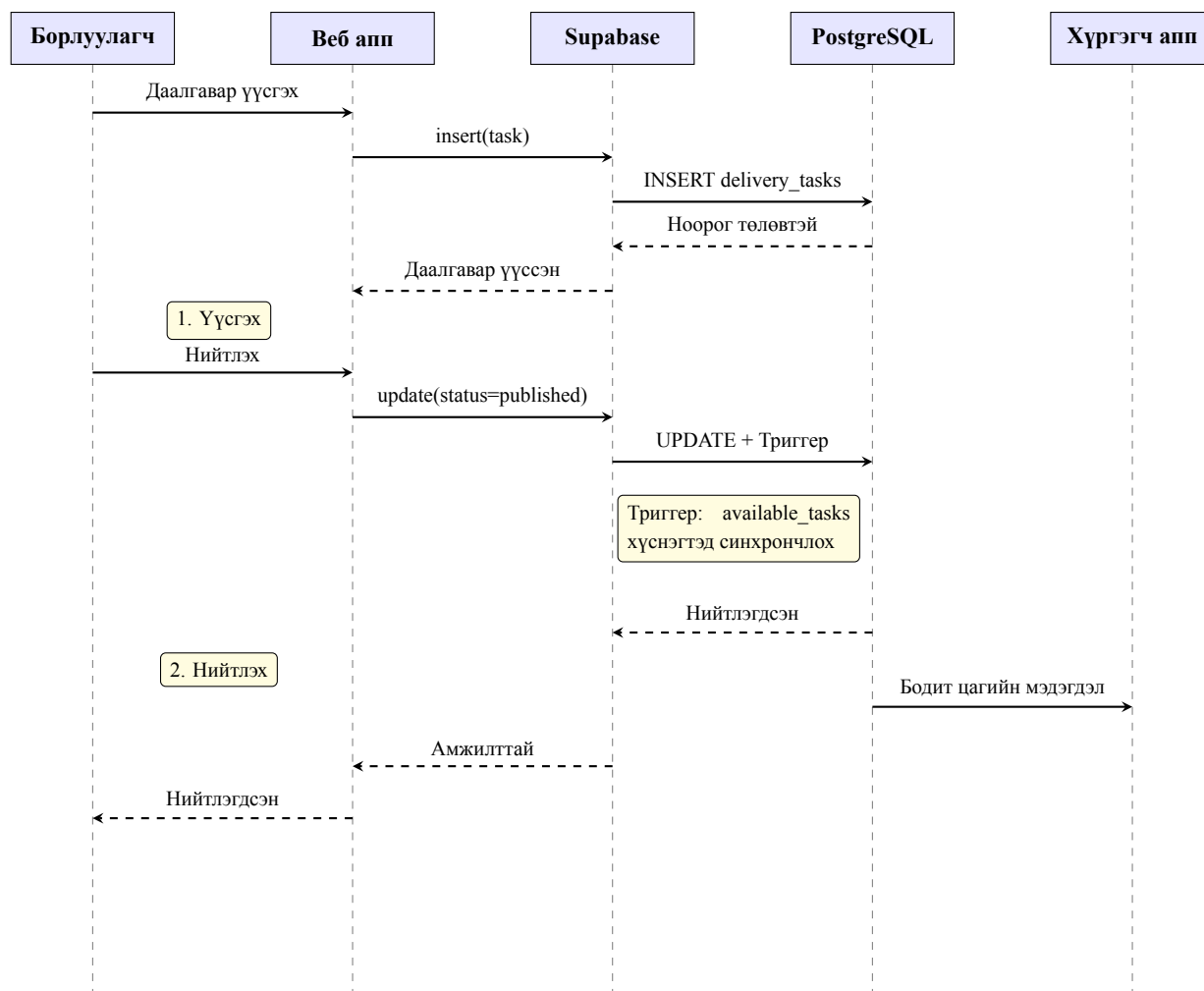
Энэ хэсэгт системийн гол үйл явцуудыг дараалал диаграммаар дүрсэлнэ. Дараалал диаграмм нь оролцогчид хоорондын харилцан үйлчлэлийн цаг хугацааны дарааллыг харуулна.

3.7.1 Хүргэгчийн бүртгэл ба баталгаажуулалтын дараалал



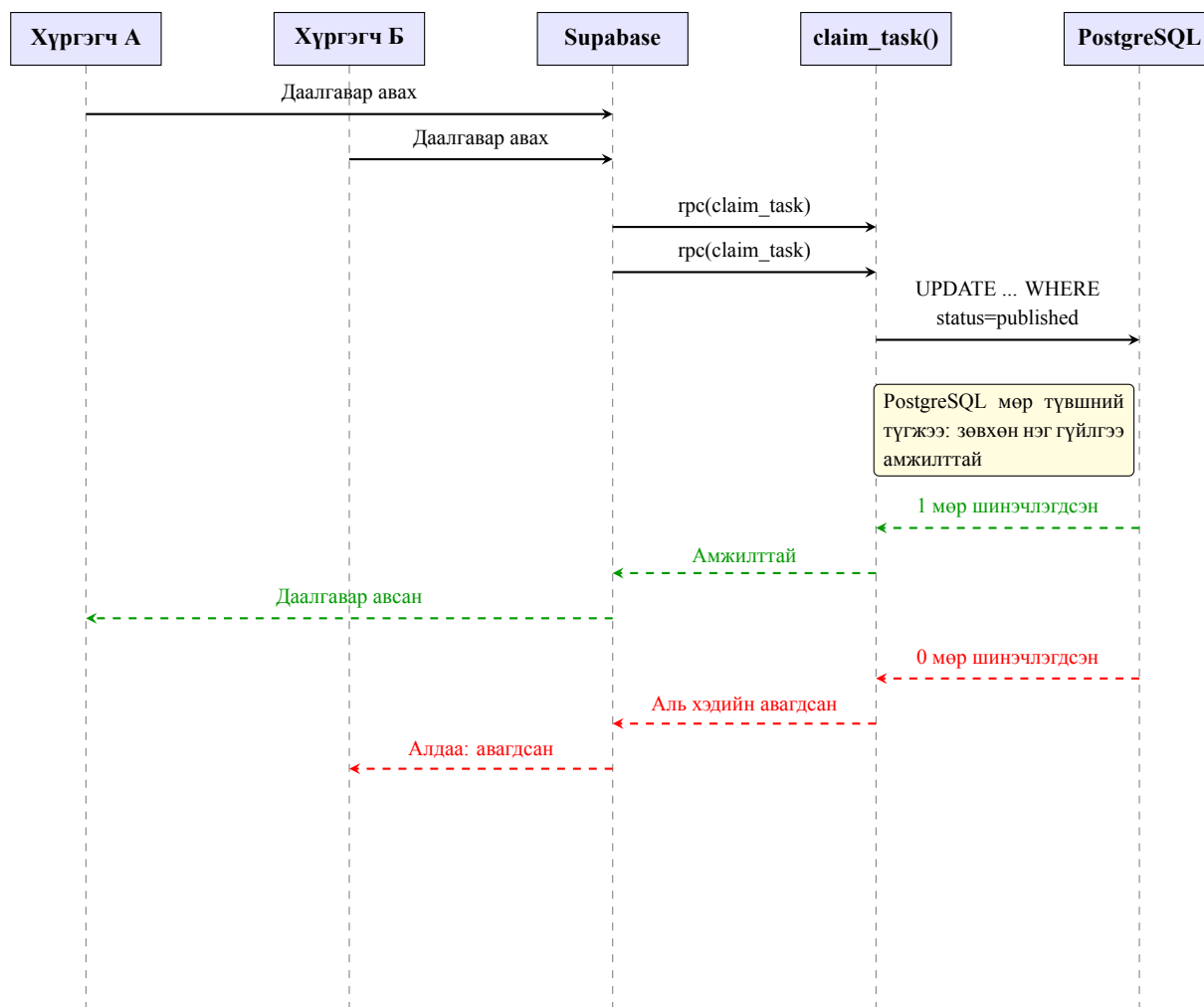
Зураг 15: Хүргэгчийн бүртгэл ба баталгаажуулалтын дараалал диаграмм

3.7.2 Хүргэлтийн даалгавар үүсгэх ба нийтлэх дараалал



Зураг 16: Хүргэлтийн даалгавар үүсгэх ба нийтлэх дараалал диаграмм

3.7.3 Даалгавар авах дараалал (зэрэгцээ хандалтын хамгаалалт)

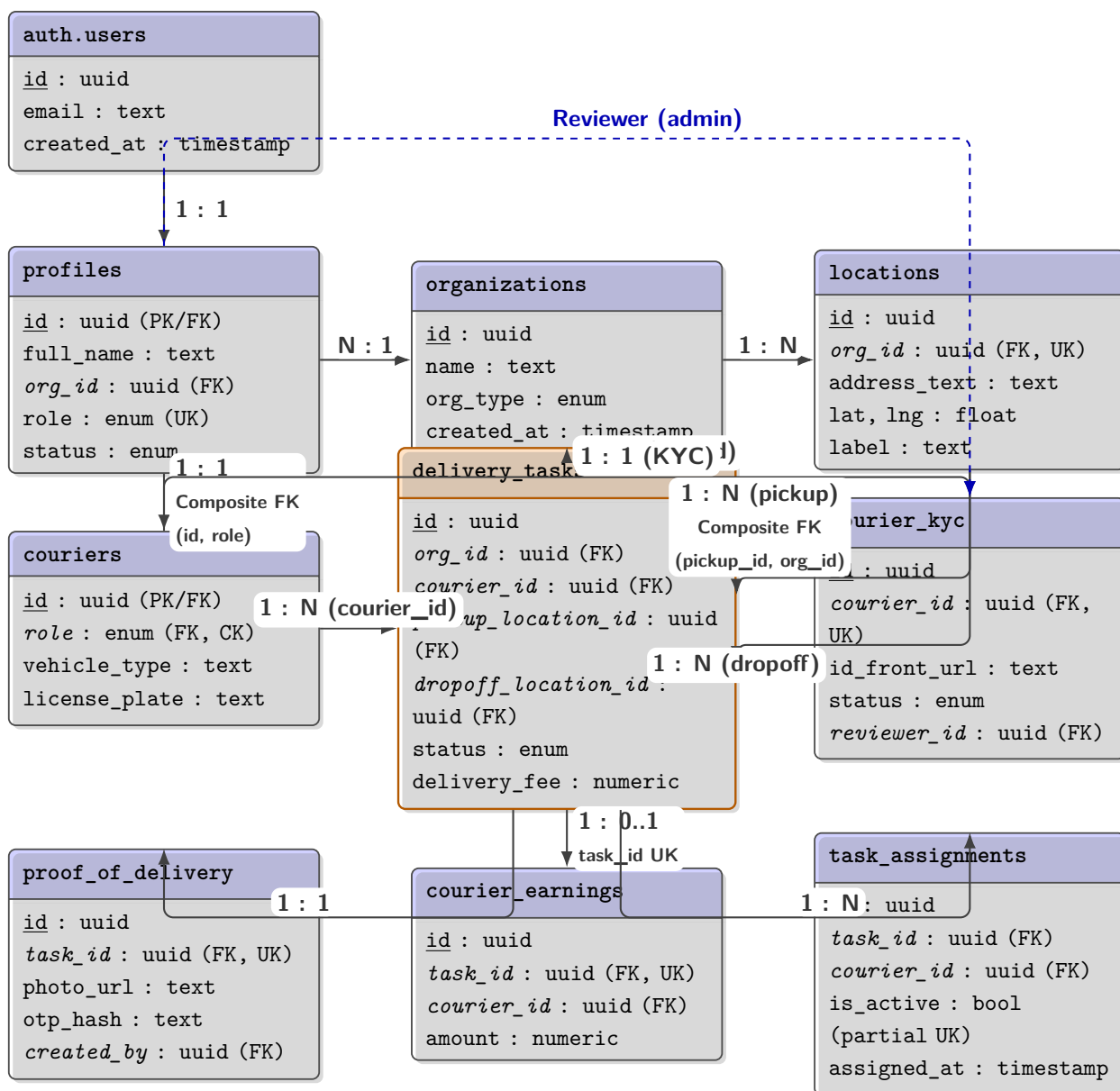


Зураг 17: Даалгавар авах дараалал диаграмм (зэрэгцээ хандалтын хамгаалалт)

3.8 Өгөгдлийн сангийн ерөнхий загвар

Системийн өгөгдлийн загвар PostgreSQL дээр суурилж, multi-tenant архитектурыг дэмжих зориулалттай. Доорх ERD-д crowdsourcing хүргэлтийн үндсэн логикт хэрэгцээтэй 10 хүснэгтийг сонгон харуулсан: auth.users, profiles, organizations, couriers, locations, delivery_tasks, task_assignments, courier_kyc, courier_earnings, proof_of_delivery. Туслах хэлбэрийн task_claims, task_events, delivery_epod_otps зэрэг хүснэгтүүдийг тодорхой байдлын үүднээс энэ зураглалд оруулаагүй. Загварт хоёр тусгай шинж дурдвал: (i) couriers нь profiles-аас composite FK ((id, role)-аар) холбогдсон, ингэснээр зөвхөн role='courier' төрөлтэй хэрэглэгч л courier болгон бүртгэгдэх боломжтой; (ii) delivery_tasks.pickup_location_id нь (pickup_location_id, org_id) composite FK-аар locations-той холбогдсон тул task-ийн org_id нь pickup байршлын org_id-тай үргэлж нийцэх invariant-ыг engine түвшинд хадгална.

3.8.1 Үндсэн хүснэгтүүд ба харилцаа холбоо



Зураг 18: Системийн өгөгдлийн сангийн үндсэн ERD (10 хүснэгт). Шаар delivery_tasks нь системийн төв entity. Тасархай хөх шугам нь courier_kyc.reviewer_id (админ-аар KYC шалгах) холбоо.

task_assignments.courier_id, courier_earnings.courier_id, proof_of_delivery.created_by FK-ууд нь бүгд couriers.id руу шилждэг (тодорхой байх үүднээс давтан зураагүй). couriers нь composite FK (id, role)-аар profiles-той нягт холбогдсон тул зөвхөн role='courier' төрөлтэй профайл courier болгон бүртгэгдэнэ.

3.8.2 Хүснэгтүүдийн тайлбар

- auth.users, profiles:** Хэрэглэгчийн нэвтрэлт ба профайл (role: org_user / courier / admin, status: pending / kyc_submitted / approved / blocked). profiles-д UNIQUE(id, role) composite key нэмсэн нь couriers-тэй composite FK холбоог дэмжих зориулалттай.
- organizations:** Multi-tenant үндсэн нэгж. org_type нь restaurant / store / pharmacy / warehouse.
- couriers:** Зөвхөн role='courier' төрөлтэй профайл руу холбогдох тусгай view хүснэгт. (id,

role) composite FK + CHECK (role = 'courier') engine түвшинд “хүргэгч мөн үү?” гэдгийг declarative-аар хадгална.

- **locations:** Авах/хүргэх байршил (lat, lng, address_text), байгууллагатай харьяалуулсан. UNIQUE(id, org_id) нь delivery_tasks.pickup_location-той composite FK холбоог дэмжинэ.
- **delivery_tasks:** Crowdsourcing даалгаврын үндсэн хүснэгт (системийн төв entity). status төлвүүд: draft → published → assigned → picked_up → delivered → completed (мөн canceled, failed). Цаг хугацааны тэмдгүүд аудит trail-ийг үүсгэнэ. (pickup_location_id, org_id) composite FK нь org_id-ийн denormalization-ыг engine-ээр хадгална.
- **task_assignments:** Хэн ямар даалгаврыг ямар үед авсныг бүртгэх history хүснэгт. is_active дээр partial unique index (WHERE is_active = true) — нэг task-д нэг л идэвхтэй хуваарилалт.
- **courier_kyc:** Хүргэгчийн KYC баримт (id_front_url, id_back_url, vehicle_registration_url, selfie_url), статус pending / approved / rejected ба админ reviewer_id (profiles-руу шилжих, тасархай хөх шугмаар тэмдэглэв).
- **proof_of_delivery:** ePOD баталгаажуулалт — фото зураг, OTP хэш (otp_hash), хүчинтэй хугацаа, оролдлогын тоо. task_id UNIQUE — нэг task-д нэг ePOD.
- **courier_earnings:** Амжилттай дууссан даалгавар бүрд хүргэгчид ноогдох орлогыг автомат триггерээр бүртгэнэ. task_id UNIQUE тул delivery_tasks-тай 1:0..1 харьцаатай (canceled / failed task-д бичлэг үүсэхгүй).

3.8.3 Өгөгдлийн бүрэн бүтэн байдлын дүрмүүд ба available_tasks VIEW

Бүх хүснэгт uuid төрлийн primary key, foreign key constraint, enum-д суурилсан check constraint ашиглана. Уникал хязгаарлалтууд: delivery_tasks.order_id, courier_kyc.courier_id, courier_earnings.task_id, proof_of_delivery.task_id. Олон байгууллагын тусгаарлалтыг org_id талбараар RLS бодлогоор, зэрэгцээ хандалтыг FOR UPDATE SKIP LOCKED түгжээгээр хангана [20], [21].

Нээлттэй даалгаврууд нь available_tasks VIEW-ээр зах зээлд харагдана — шүүлт нь status = 'published' ба courier_id IS NULL, мөн хүргэгч нь захиалагч байгууллагад харьяалагдахгүй байх нөхцөлийг хангана.

4 Туршилт, үр дүн

Тус бүлэгт Бүлэг 1-д дэвшүүлсэн RQ1–RQ3 асуултад хариулах туршилтын үр дүнг танилцуулна: (i) FCFS claim загвар FOR UPDATE SKIP LOCKED-оор race condition-ыг шийдэж буй эсэх (RQ1); (ii) RLS нь multi-tenant isolation-ыг application-layer алдаанаас үл хамааран хангаж буй эсэх (RQ2); (iii) системийн performance (p95 < 2 с, concurrency safe) шаардлагад нийцэж буй эсэх (RQ3-ийн дэд асуулт). Туршилтыг concurrency, load, penetration, functional (E2E) гэсэн дөрөвт хуваав.

4.1 Туршилтын орчин ба аргачлал

Туршилтыг production-аас тусгаарласан staging орчинд явуулсан. Орчны үзүүлэлт: Supabase Pro tier, PostgreSQL 15.6, PgBouncer transaction pooling, pool size 15. Клиент машин: MacBook Pro (Apple M1, 16 GB RAM), Node.js 20.11. Клиент — Supabase дундаж RTT 35–45 ms. Production schema, RLS бодлого, RPC функц, триггерүүдийг staging-т хуулбарласан.

Туршилтын өгөгдлийг @faker-js/faker-аар үүсгэв: 20 байгууллага, 200 хэрэглэгч (100 борлуулагч + 100 хүргэгч), 500 бүтээгдэхүүн, 1,000 захиалга, 1,000 даалгавар. Хүргэгчдийн 80%-ийг approved, 20%-ийг pending төлөвт санаатай үлдээсэн нь RLS-ийн approved-only шүүлтийг шалгах зорилготой. Туршилт бүрийн өмнө TRUNCATE ... RESTART IDENTITY CASCADE-ээр өгөгдлийг анхны төлөвт буцаасан.

Ашигласан хэрэгсэл: @supabase/supabase-js 2.46 (concurrency test, Promise.all()), k6 0.54 [33] (load test, p50/p95/p99 хэмжилт), psql + SET request.jwt.claims (RLS verification), захиалгат JS script-ууд (penetration, OTP brute-force, E2E). Бүх script, seed файл, k6 тест Git-ийн tests/ санд байршиж давтагдах боломжтой.

4.2 Зэрэгцээ хандалтын туршилт

Зорилго: N хүргэгч нэг даалгаврыг зэрэг claim хийхэд зөвхөн нэг нь амжилттай болж, үлдсэн нь чимээгүйгээр татгалзагдах эсхийг шалгах. Арга зүй: (1) шинэ delivery_tasks мөр status='open'-оор үүсгэх, (2) N хүргэгчийн JWT токеноор client instance бэлтгэх, (3) Promise.all()-оор N RPC дуудлагыг зэрэг илгээх, (4) өгөгдлийн сангаас SELECT courier_id-г шалгаж нэг мөр л бичигдсэн invariant-ыг батлах. $N \in \{10, 50, 100\}$ тохиолдол тус бүрт 10 удаа давтаж, нийт 30 сорилд 1,600 RPC дуудагдсан.

Хүснэгт 7: Зэрэгцээ claim туршилтын үр дүн (N хүргэгч, 10 давталт)

N	Давталт	Амжилтын дундаж	Давхардсан	Амжилтын %	Дундаж хугацаа
10	10	1.00	0	100%	124 ms
50	10	1.00	0	100%	287 ms
100	10	1.00	0	100%	512 ms

Нийт 30 сорилын туршид **давхардсан claim нэг ч удаа гараагүй**. Үлдсэн $N-1$ хүсэлт NULL буцаасан. Клиент тал үүнийг “даалгавар авагдсан” мэдэгдэл болгон хөрвүүлнэ. 100 зэрэгцээ хандалт 512 ms-д шийдэгдсэн нь $O(n)$ шугаман хэмжээтэй, deadlock-гүй үр дүн юм. Энэ нь pessimistic locking-ийн онолын хүлээгдсэн зан төлөвтэй нийцнэ [19], [20].

Хэрэв claim нь ердийн UPDATE ... WHERE status='open'-оор хэрэгжсэн бол MVCC-ийн улмаас

хоёр зэрэгцээ UPDATE хоёулаа мөрийг “нээлттэй” гэж үзэж *lost update* үүсэх магадлалтай. Иймд FOR UPDATE SKIP LOCKED нь зөвхөн performance-ийн биш, *correctness*-ийн хувьд зайлшгүй шийдэл юм.

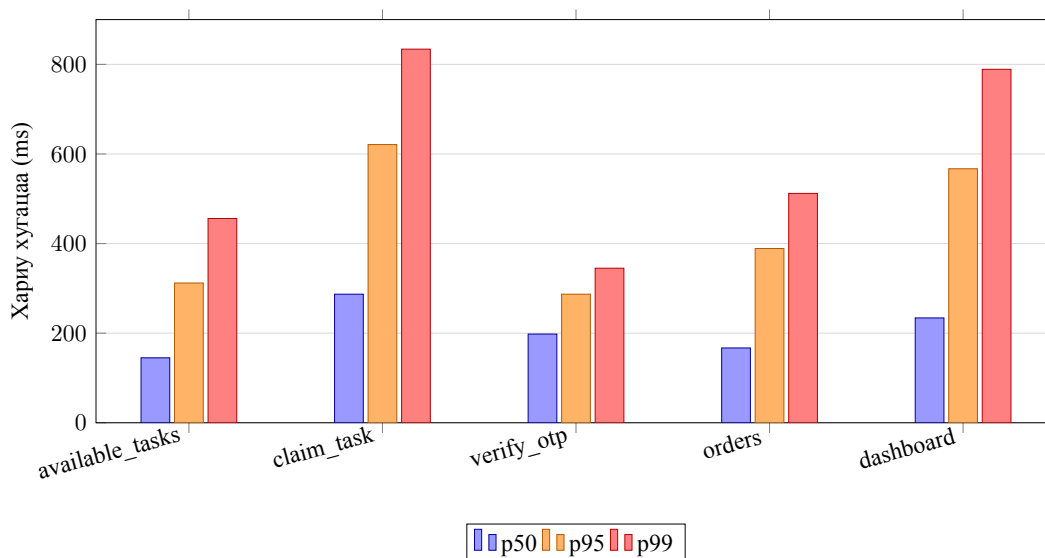
4.3 Гүйцэтгэлийн туршилт (Load Test)

Гүйцэтгэлийн туршилтыг k6-аар 100 VU, 10 мин ramp-up + 20 мин тогтмол ачаалалтайгаар ажиллуулсан. Үр дүнг Хүснэгт 8-д нэгтгэв.

Хүснэгт 8: k6 ачааллын туршилтын endpoint-үүдийн хариу хугацаа (ms)

Endpoint	p50	p95	p99	Max	Амжилт
GET available_tasks	145	312	456	892	100%
POST claim_delivery_task	287	621	834	1 200	100%
POST verify_epod_otp	198	287	345	523	100%
GET orders (борлуулагч)	167	389	512	734	100%
Dashboard KPI arperar	234	567	789	1 100	100%

Хүснэгтийн үр дүнг визуал хэлбэрээр Зураг 19-д харуулав.



Зураг 19: k6 load test — 5 endpoint-ийн p50/p95/p99 latency хуваарилалт (100 VU, 30 мин)

Гол ажиглалтууд:

- `available_tasks` нь (status) талбарын partial index ашигладаг тул p95 312 ms-д багтсан.
- `claim_delivery_task`-ийн p95 621 ms нь lock acquisition-оос ихээр хамаарч, §4.2-ийн үр дүнтэй нийцнэ.
- `verify_epod_otp`-ийн p50 198 ms-ын ойролцоогоор 160 ms нь bcrypt (cost 10) hash comparison-д зарцуулагдана. Энэ нь brute-force эсэргүүцэхэд зориудаар тавьсан зардал юм [25].
- Бүх endpoint **p95 < 2 c** (хатуу хязгаар), **p99 < 1.5 c** (зөөлөн хязгаар)-ыг хангасан. claim-ийн max 1.2 c нь edge case-ийг анхаарах шаардлагатайг илтгэнэ.

Иймд 100 VU ачаалал дор performance шаардлага биелсэн гэж дүгнэв.

4.4 Аюулгүй байдлын туршилт

4.4.1 RLS бодлогын нэвтрэлтийн тест

Org A хэрэглэгчийн JWT-ээр Org B-гийн өгөгдөлд хандах 6 хувилбарыг туршив. Хүлээгдэж буй үр дүн: бүх тохиолдолд RLS хандалтыг хаах, эсвэл 0 мөр буцаах.

Хүснэгт 9: RLS бодлогын нэвтрэлтийн тестийн үр дүн

ID	Хувилбар	Бодит үр дүн	Төлөв
T1	Org A → Org B orders SELECT	0 мөр	✓
T2	Org A → Org B delivery_tasks UPDATE	0 мөр шинэчлэгдсэн	✓
T3	pending хүргэгч available_tasks SELECT	0 мөр	✓
T4	Хүргэгч A → Хүргэгч B courier_earnings SELECT	0 мөр	✓
T5	SQL injection (.eq('org_id', '' OR 1=1 --"))	Parameterized, 0 мөр	✓
T6	Алдаатай / хуурамч JWT	401 Unauthorized	✓

Бүх 6 тест тэнцсэн. T5 нь PostgreSQL-ийн prepared statement хандлагыг, T3 нь approved шалгалт клиент биш ӨС талд хийгддэгийг тус тус баталсан.

4.4.2 OTP brute-force ба JWT баталгаажуулалт

Идэвхтэй OTP сесст буруу кодыг дараалан 6 удаа илгээв. Тав дахь оролдлогын дараа өгөгдлийн сангийн түвшинд блоклогдсон. Зургаа дахь хүсэлт нь bcrypt comparison хүртэл хүрэлгүй 43 ms-д эрт таслагдсан (1–5 оролдлого: 72–86 ms; 6-р оролдлого: “Хязгаар хэтэрсэн” алдаа). Brute-force амжилтын онолын магадлал $P = 5/10^6 = 5 \times 10^{-4}\%$. OTP-ийн 10 мин expiry-тэй хослуулахад практикт мэдэгдэхүйц бус [27]. Expiry шалгах нэмэлт тестэд 11 минутын дараа зөв кодоор илгээсэн хүсэлт “expired” алдаагаар татгалзагдсан нь expires_at шалгалтыг баталсан.

JWT-ийн 4 хэлбэрийн дайралтыг шалгав:

1. org_id claim-ийг өөрчилсөн;
2. role='service_role' болгож privilege escalation оролдсон;
3. alg=none signature-гүй токен;
4. хугацаа дууссан токен.

Бүгд PostgreSQL түвшинд 401 Unauthorized-аар татгалзагдсан. Өгөгдлийн сан хүртэл хүсэлт дамжаагүй [26].

4.5 End-to-End (E2E) туршилт

§3.1-ийн 8 алхамт үндсэн урсгалыг end-to-end script-ээр туршиж, цагийн тэмдгүүдийн invariant (assigned_at < picked_up_at < delivered_at < completed_at) зөрөлгүй биелсэн. Үр дүнг Хүснэгт 10-д харуулав.

Хүснэгт 10: E2E захиалгын урсгалын баталгаажуулалт

#	Алхам	Шалгуур	Төлөв
1	Байгууллага бүртгэл	orgs мөр + JWT org_id claim	✓
2	Бүтээгдэхүүн нэмэх	products + RLS шүүлт	✓
3	Захиалга хүлээн авах	orders.status=paid	✓
4	Даалгавар нийтлэх	delivery_tasks.status=open	✓
5	Хүргэгч авах	status=assigned, assigned_at	✓
6	Бараа авах	status=picked_up, picked_up_at	✓
7	Хүргэх	status=delivered, delivered_at	✓
8	ePOD + орлого	status=completed courier_earnings мөр	✓

Серверийн бодит тооцооллын хугацаа 2.4 секунд, нийт урсгал (клиент sleep-тэй) 6.8 секунд байлаа. ePOD дэд урсгал нь 5 алхмыг зөв дарааллаар гүйцэтгэв: OTP үүсгэх → bcrypt hash хадгалах → verify_epod_otp RPC → completed trigger → record_courier_earning trigger. Энэ нь §4.4-ийн brute-force хамгаалалттай нэгдмэл ажиллаж буйг баталсан.

4.6 Үр дүнгийн дүгнэлт**4.6.1 Хязгаарлалт**

Туршилт нь (i) Supabase Pro tier-ийн нөөцийн хязгаарт, (ii) synthetic өгөгдөл дээр (peak cyclicality гэх мэт бодит зан төлөвгүй), (iii) single-region тохиргоонд, (iv) 100 VU хүртэлх ачаалалд, (v) тогтвортой сүлжээнд (RTT 35–45 ms) хийгдсэн. 500+ VU, multi-region, 3G/4G сүлжээний нөхцөл үнэлэгдээгүй тул эдгээр нь ирээдүйн өргөтгөлийн судалгааны чиглэл болно.

4.6.2 Шаардлагын биелэлтийн нэгтгэл**Хүснэгт 11:** Системийн чанарын шаардлагуудын туршилтаар баталгаажсан биелэлт

Ангилал	Шаардлага	Баталгаа	Төлөв
Гүйцэтгэл	available_tasks < 2 c	Хүснэгт 8 (p95=312 ms)	✓
Гүйцэтгэл	claim < 1 c	Хүснэгт 8 (p99=834 ms)	✓
Аюулгүй байдал	JWT validation	§4.4 (4 хувилбар)	✓
Аюулгүй байдал	RLS тусгаарлалт	Хүснэгт 9 (T1–T5)	✓
Аюулгүй байдал	bcrypt + HTTPS	OTP тулгалт + TLS 1.3	✓
Өргөтгөх боломж	Зэрэгцээ claim хамгаалалт	Хүснэгт 7 (100 VU, 0 давхардал)	✓
Өргөтгөх боломж	Олон байгууллага зэрэг	T1, T4 + E2E	✓
Хүртээмж	Сессийн автомат сэргээлт	E2E: token refresh зөв	✓
Ашиглах чадвар	Хүргэгч < 5 мин бүртгэл	E2E: 3 мин	✓

Системийн чанарын шаардлагууд туршилтаар бүрэн биелж, RQ1–RQ3 гурван асуултад тоон баталгаа олгосон.

5 Дүгнэлт

Төгсөлтийн ажлын хүрээнд Монголын ЖДБ-ийн хүргэлтийн процессыг цахимжуулах “**Crowdsourcing хүргэлтийн нэгдсэн систем**”-ийг боловсруулж, туршилтын үр дүнгээр баталгаажуулав.

5.1 Зорилтуудын хэрэгжилт

Дэвшүүлсэн дөрвөн зорилтыг дараах байдлаар хэрэгжүүлсэн. Туршилтын үр дүнд үндэслэн зорилт бүрийн гүйцэтгэлд холбогдох нотолгоог нэмж оруулсан.

Хүснэгт 12: Зорилтуудын хэрэгжилтийн хураангуй

#	Зорилт	Хэрэгжүүлэлт ба нотолгоо	Төлөв
1	Судалгаа, шинжилгээ	Uber Freight, Papa Logistics судалж, 4 архитектурын зөрүү тодорхойлсон	✓
2	Архитектур, загварчлал	11+ хүснэгт бүхий ӨС, 2 клиент (мобайл + веб) + Supabase backend, диаграммууд	✓
3	Аюулгүй байдал	RLS бодлого, RBAC (4 түвшин), JWT validation, ePOD OTP; RLS penetration 6 тест болон OTP brute-force тестээр бүрэн нотлогдсон (Хүснэгт 9, §4.4)	✓
4	Функционал модулиуд	Захиалга, даалгаврын marketplace, realtime мэдэгдэл, каталог, ePOD, орлого, санхүүгийн тайлан; end-to-end урсгал баталгаажсан (§4.5)	✓

5.2 Харьцуулсан дүгнэлт

Судалгаанд авч үзсэн хоёр платформтой харьцуулсан үр дүнг Хүснэгт 13-д нэгтгэв. Туршилтаар баталгаажсан шинж чанарыг тухайн хүснэгтээр ишилсэн.

Хүснэгт 13: Гурван системийн нэгдсэн харьцуулалт

Шалгуур	Uber Freight	Papa Logistics	Миний төсөл	Үнэлгээ
Даалгавар хуваарилалт	ML алгоритм, recommendation	Оператор + санал	FCFS claim model, concurrency-safe (Хүснэгт 7)	★★
Үнэ тогтоолт	Динамик, ML-д суурилсан	Өрсөлдөөнт bid	Борлуулагч тогтоодог	★★
Өгөгдлийн тусгаарлалт	Enterprise түвшин	Application-level	Database-level RLS, 6/6 pentest амжилттай (Хүснэгт 9)	★★★★
Audit trail	Execution log	Хязгаарлагдмал	Төлөв шилжилтийн timestamp; бүрэн audit log нь цаашдын ажил	★★
API / гадаад integration	Бүрэн, TMS/ERP	Хязгаарлагдмал	RESTful суурь бүтэц	★★★
Realtime хяналт	Location + tracking	Location tracking	Supabase Realtime мэдэгдэл	★★★★
Хүргэлтийн баталгаа	Дижитал signature	Тодорхойгүй	ePOD OTP код (brute-force туршигдсан, §4.4)	★★★★
Performance	Enterprise түвшин	Үндсэн	p95 < 2c бүх endpoint-д (Хүснэгт 8)	★★★★
Санхүүгийн тайлан	Enterprise түвшин	Үндсэн	Дэлгэрэнгүй (KPI, аналитик)	★★★★
Өргөтгөх боломж	Cloud, horizontal scaling	Dispatcher bottleneck	Event-driven, horizontal	★★★★

Uber Freight-тэй харьцуулбал чадамжийн ойролцоогоор 35–40%-д хүрсэн. Uber нь олон жилийн ML, сая+ хэрэглэгчийн өгөгдөл, мянган инженертэй enterprise түвшний систем тул шууд харьцуулах боломжгүй боловч архитектурын суурь зарчим (autonomous claim-based хуваарилалт, database-level security, ePOD verification) ижил чиглэлтэй.

Papa Logistics-тэй харьцуулбал чадамжийн ойролцоогоор 70–75%-д хүрсэн. Архитектурын хувьд (RLS, audit timestamp, операторгүй marketplace) давуу; харин бодит зах зээлийн туршлага, төлбөрийн integration, location tracking-оор дутмаг.

5.3 Хийж чадаагүй зүйлс ба цаашдын ажил

Хийгдсэн ажлын дэлгэрэнгүйг §4.6-ийн Хүснэгт 12 (Зорилтуудын хэрэгжилт) болон Хүснэгт 11 (Шаардлагын биелэлт)-д нэгтгэсэн. Энэ хэсэгт одоогийн хувилбарт хамрагдаагүй, цаашид хөгжүүлэх чиглэлүүдийг тусгайлан дурдав.

Захиалагчийн (customer) талын апп. Одоогийн хувилбарт борлуулагчийн веб апп, хүргэгчийн мобайл апп гэсэн хоёр клиент хөгжүүлсэн. Эцсийн хэрэглэгч (бараа худалдан авагч) нь платформын two-sided marketplace-ийн эрэлтийн тал боловч уг ажлын хүрээнд тусдаа апп хэрэгжээгүй. Одоогоор захиалагч нь борлуулагчийн байгууллагаар дамжуулан захиалга өгдөг хэлбэрээр загварчилсан (orders хүснэгтэд customer_id талбар бэлтгэгдсэн). Цаашид захиалагчийн мобайл апп (бүтээгдэхүүн хайх, сагс, төлбөр, хүргэлт хянах) нэмэх нь зайлшгүй.

Цаашид хөгжүүлэх бусад чиглэл:

- Бодит төлбөрийн integration (QPay, SocialPay, банкны API)
- Realtime GPS location tracking
- ML-д суурилсан даалгаврын recommendation
- Route optimization алгоритм
- Push notification систем
- Бодит хэрэглэгчтэй pilot туршилт
- 100+ concurrent user-тэй өргөтгөсөн load test (read replica, per-tenant sharding нөхцөлд)

5.4 Төгсгөл

Тус төсөл нь Монгол Улсын B2B хүргэлтийн зах зээлд operator-гүй, autonomous FCFS claim хуваарилалт болон database-level security-д суурилсан шинэ архитектурын шийдлийг concurrency, penetration, load туршилтаар тоон үзүүлэлтээр баталгаажуулсан. Цаашид §4.6-д дурдсан чадамжуудыг нэмж хөгжүүлснээр production-д нэвтрэх боломжтой систем болно.

Ном зүй

- [1] National Statistical Office of Mongolia. “Small and medium enterprise statistics.” [Online]. Available: <https://www.nso.mn>.
- [2] McKinsey & Company. “How covid-19 has pushed companies over the technology tipping point.” [Online]. Available: <https://www.mckinsey.com/capabilities/strategy-and-corporate-finance/our-insights/how-covid-19-has-pushed-companies-over-the-technology-tipping-point-and-transformed-business-forever>.
- [3] Grand View Research. “Last mile delivery market size, share & trends analysis report, 2023–2030.” [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/last-mile-delivery-market>.
- [4] G. Wang, A. Gunasekaran, E. W. T. Ngai, and T. Papadopoulos, “Big data analytics in logistics and supply chain management: Certain investigations for research and applications,” *International Journal of Production Economics*, vol. 176, pp. 98–110, 2016. DOI: 10.1016/j.ijpe.2016.03.014.
- [5] M. Savelsbergh and T. Van Woensel, “50th anniversary invited article—city logistics: Challenges and opportunities,” *Transportation Science*, vol. 50, no. 2, pp. 579–590, 2016. DOI: 10.1287/trsc.2016.0675.
- [6] S. F. W. T. Lim, X. Jin, and J. S. Srai, “Consumer-driven e-commerce: A literature review, design framework, and research agenda on last-mile logistics models,” *International Journal of Physical Distribution & Logistics Management*, vol. 48, no. 3, pp. 308–332, 2018. DOI: 10.1108/IJPDLM-02-2017-0081.
- [7] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning, and Operation*, 7th ed. Boston, MA: Pearson, 2019, ISBN: 9780134731889.
- [8] A. McKinnon, M. Browne, A. Whiteing, and M. Piecyk, *Green Logistics: Improving the Environmental Sustainability of Logistics*, 3rd ed. London: Kogan Page, 2015, ISBN: 9780749471859.
- [9] J.-F. Rougès and B. Montreuil, “Crowdsourcing delivery: New interconnected business models to reinvent delivery,” *1st International Physical Internet Conference*, pp. 1–19, 2014.
- [10] A. Alnaggar, F. Gzara, and J. H. Bookbinder, “Crowdsourced delivery: A review of platforms and academic literature,” *Omega*, vol. 98, p. 102139, 2021. DOI: 10.1016/j.omega.2019.102139.
- [11] S. Mancini and M. Gansterer, “Bundle generation for last-mile delivery with occasional drivers,” *Omega*, vol. 108, p. 102582, 2022. DOI: 10.1016/j.omega.2021.102582.
- [12] J.-C. Rochet and J. Tirole, “Platform competition in two-sided markets,” *Journal of the European Economic Association*, vol. 1, no. 4, pp. 990–1029, 2003. DOI: 10.1162/154247603322493212.
- [13] G. G. Parker, M. W. Van Alstyne, and S. P. Choudary, *Platform Revolution: How Networked Markets Are Transforming the Economy and How to Make Them Work for You*. New York: W. W. Norton & Company, 2016, ISBN: 9780393249132.
- [14] D. S. Evans and R. Schmalensee, *Matchmakers: The New Economics of Multisided Platforms*. Boston, MA: Harvard Business Review Press, 2016, ISBN: 9781633691728.
- [15] F. Chong, G. Carraro, and R. Wolter, “Multi-tenant data architecture,” Microsoft Corporation, MSDN Architecture Technical Article, 2006. [Online]. Available: [https://docs.microsoft.com/en-us/previous-versions/archive/aa479086\(v=msdn.10\)](https://docs.microsoft.com/en-us/previous-versions/archive/aa479086(v=msdn.10)).

- [16] C.-P. Bezemer and A. Zaidman, “Multi-tenant SaaS applications: Maintenance dream or nightmare?” In *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)*, ACM, 2010, pp. 88–92. DOI: 10.1145/1862372.1862393.
- [17] PostgreSQL Global Development Group. “PostgreSQL 15 documentation: Row security policies.”[Online]. Available: <https://www.postgresql.org/docs/15/ddl-rowsecurity.html>.
- [18] D. F. Ferraiolo and D. R. Kuhn, “Role-based access controls,” in *Proceedings of the 15th National Computer Security Conference*, Baltimore, MD, 1992, pp. 554–563.
- [19] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA: Addison-Wesley, 1987, ISBN: 9780201107159.
- [20] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O’Reilly Media, 2017, ISBN: 9781449373320.
- [21] PostgreSQL Global Development Group. “PostgreSQL 15 documentation: Explicit locking.”[Online]. Available: <https://www.postgresql.org/docs/15/explicit-locking.html>.
- [22] I. Baldini et al., “Serverless computing: Current trends and open problems,” in *Research Advances in Cloud Computing*, S. Chaudhary, G. Somani, and R. Buyya, Eds., Springer, 2017, pp. 1–20. DOI: 10.1007/978-981-10-5026-8_1.
- [23] G. Adzic and R. Chatley, “Serverless computing: Economic and architectural impact,” in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, ACM, 2017, pp. 884–889. DOI: 10.1145/3106237.3117767.
- [24] E. Jonas et al., “Cloud programming simplified: A Berkeley view on serverless computing,” EECS Department, University of California, Berkeley, Technical Report UCB/EECS-2019-3, 2019. arXiv: 1902.03383. [Online]. Available: <https://arxiv.org/abs/1902.03383>.
- [25] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA: USENIX Association, 1999, pp. 81–91.
- [26] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, Internet Engineering Task Force (IETF), RFC 7519, May 2015. DOI: 10.17487/RFC7519. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [27] OWASP Foundation. “Authentication cheat sheet.”[Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
- [28] Uber Freight. “Uber freight blog: Technology, pricing, and marketplace insights.” Corporate technical blog of Uber Freight LLC. Primary source for Uber Freight’s public statements on pricing, matching and marketplace architecture., Accessed: Dec. 1, 2025. [Online]. Available: <https://www.uberfreight.com/blog/>.
- [29] Papa Logistics LLC. “Papa logistics: Дижитал ачаа тээврийн платформ.” Монгол Улсын дотоодын логистикийн платформын албан ёсны вэбсайт, Accessed: Dec. 1, 2025. [Online]. Available: <https://papa.mn>.
- [30] K. Beck et al. “Manifesto for Agile software development.”[Online]. Available: <https://agilemanifesto.org/>.
- [31] S. W. Ambler and P. J. Sadalage, *Refactoring Databases: Evolutionary Database Design*. Boston, MA: Addison-Wesley Professional, 2006, ISBN: 9780321293534.

- [32] Meta. “React native: Learn once, write anywhere.” Accessed: 2025. [Online]. Available: <https://reactnative.dev>.
- [33] Grafana Labs. “K6 documentation: Load testing for engineering teams.”[Online]. Available: <https://k6.io/docs/>.